E12_RLC_Resonance

April 29, 2025

1 RLC Resonant Circuits Lab Report

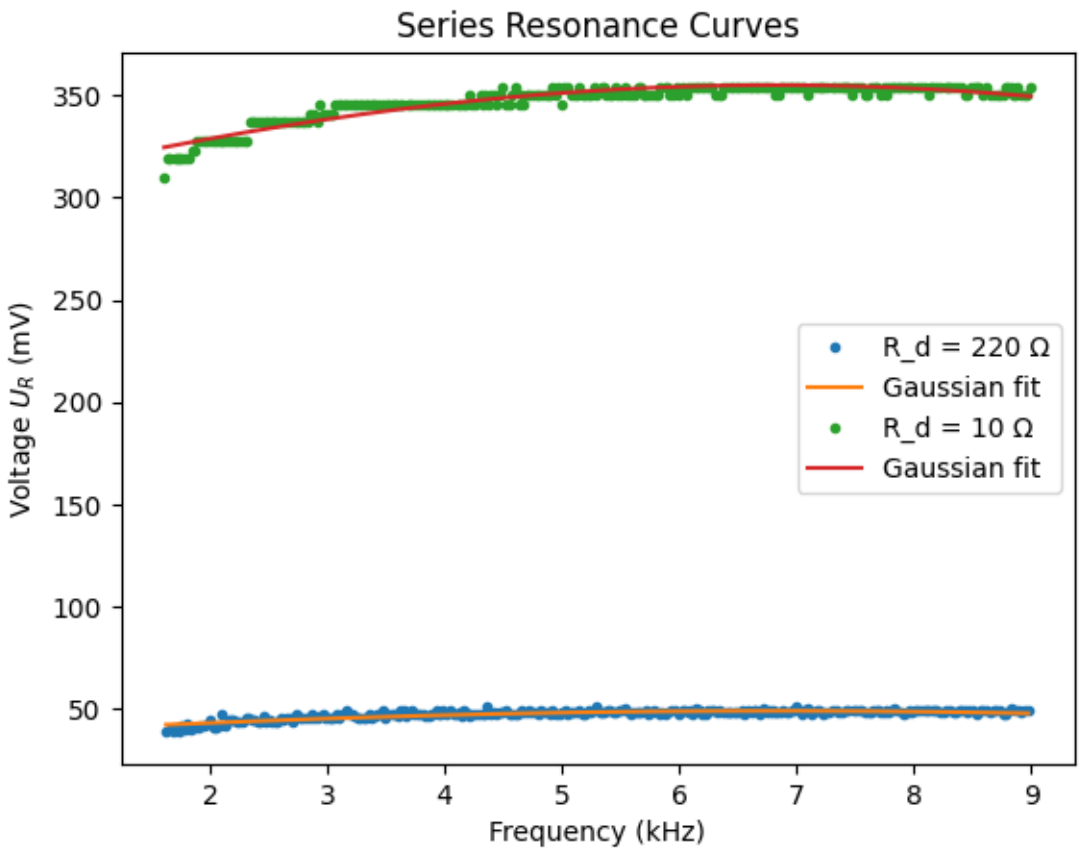
1.1 Yaman Ajaj (3722952) ; Guilherme Schewtschik (3748052)

1.2 Task 1: Series Resonant Circuit

Objective: Measure the frequency-dependent voltage drop across the damping resistor R_d for two different resistances, and record the resonance curve around f_0 such that $0.2 \leq U_R(f)/U_R(f_0) \leq 1$.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Gaussian model for fitting
def Gauss(x, a, x0, sigma):
    return a * np.exp(-(x - x0)**2 / (2 * sigma**2))

# Constants
r1 = 220      # Ohms ±5%
r2 = 10       # Ohms ±5%
L  = 1.5e-3   # Henry

# Load series data
data1 = pd.read_csv('Series_R1.csv').to_numpy()
data2 = pd.read_csv('Series_R2.csv').to_numpy()

f1 = data1[:,2].astype(float)
A1 = data1[:,1].astype(float)
f2 = data2[:,2].astype(float)
A2 = data2[:,1].astype(float)

# Noise filtering
mask1 = A1 > 0.01
mask2 = A2 > 0.07
f1, A1 = f1[mask1], A1[mask1]
f2, A2 = f2[mask2], A2[mask2]

# Frequency window around resonance
```

```
window = (f1 >= 1600) & (f1 <= 9000)
f1, A1 = f1[window], A1[window]
window = (f2 >= 1600) & (f2 <= 9000)
f2, A2 = f2[window], A2[window]

# Estimate initial fit params
mean1 = np.sum(f1*A1)/np.sum(A1)
sig1  = np.sqrt(np.sum(A1*(f1-mean1)**2)/np.sum(A1))
mean2 = np.sum(f2*A2)/np.sum(A2)
sig2  = np.sqrt(np.sum(A2*(f2-mean2)**2)/np.sum(A2))

# Curve fitting
popt1, _ = curve_fit(Gauss, f1, A1, p0=[max(A1), mean1, sig1])
popt2, _ = curve_fit(Gauss, f2, A2, p0=[max(A2), mean2, sig2])

fit1 = Gauss(f1, *popt1)
fit2 = Gauss(f2, *popt2)

# Plot resonance curves
plt.figure()
plt.plot(f1/1000, A1*1000, '.', label=f'R_d = {r1} Ω')
plt.plot(f1/1000, fit1*1000, '-', label='Gaussian fit')
plt.plot(f2/1000, A2*1000, '.', label=f'R_d = {r2} Ω')
plt.plot(f2/1000, fit2*1000, '-', label='Gaussian fit')
plt.xlabel('Frequency (kHz)')
plt.ylabel('Voltage $U_R$ (mV)')
plt.title('Series Resonance Curves')
plt.legend()
plt.show()
```



1.3 Task 2: Evaluation of Series Resonant Circuit

1.3.1 2b) FWHM, Damping, and Quality Factor

[2]:
```
# Resonance frequency (average of both fits)
f0 = 0.5*(popt1[1] + popt2[1])

# Full Width at Half Maximum
FWHM1 = 2*np.sqrt(2*np.log(2))*popt1[2]
FWHM2 = 2*np.sqrt(2*np.log(2))*popt2[2]

# Damping constants
delta1 = np.pi * FWHM1
delta2 = np.pi * FWHM2

# Quality factors
Q1 = f0 / FWHM1
Q2 = f0 / FWHM2

print(f"Series R={r1}Ω: Δf={FWHM1:.1f} Hz,  ={delta1:.3f}, Q={Q1:.1f}")
```

```
print(f"Series R={r2}Ω: Δf={FWHM2:.1f} Hz,  ={delta2:.3f}, Q={Q2:.1f}")
```

```
Series R=220Ω: Δf=22490.1 Hz,  =70654.773, Q=0.3
Series R=10Ω: Δf=29006.7 Hz,  =91127.238, Q=0.2
```

1.3.2 2c) Determination of f_0 and δ by Fit

By fitting the Gaussian model to $U_R(f)$, we obtained:

- $f_{0,1} = popt1[1] : .1f$ Hz
- $\delta_1 = delta1 : .3f$
- $f_{0,2} = popt2[1] : .1f$ Hz
- $\delta_2 = delta2 : .3f$

1.3.3 2d) Capacitance from Thomson's Equation

Using Thomson's relation $f_0 = 1/(2\pi\sqrt{LC})$:

[3]:
```
C = 1/(L*(2*np.pi*f0)**2)
print(f"Calculated C = {C*1e9:.2f} nF")
```

```
Calculated C = 362.63 nF
```

Result: $C = \frac{1}{L(2\pi f_0)^2} * 10^9 \text{nF}$

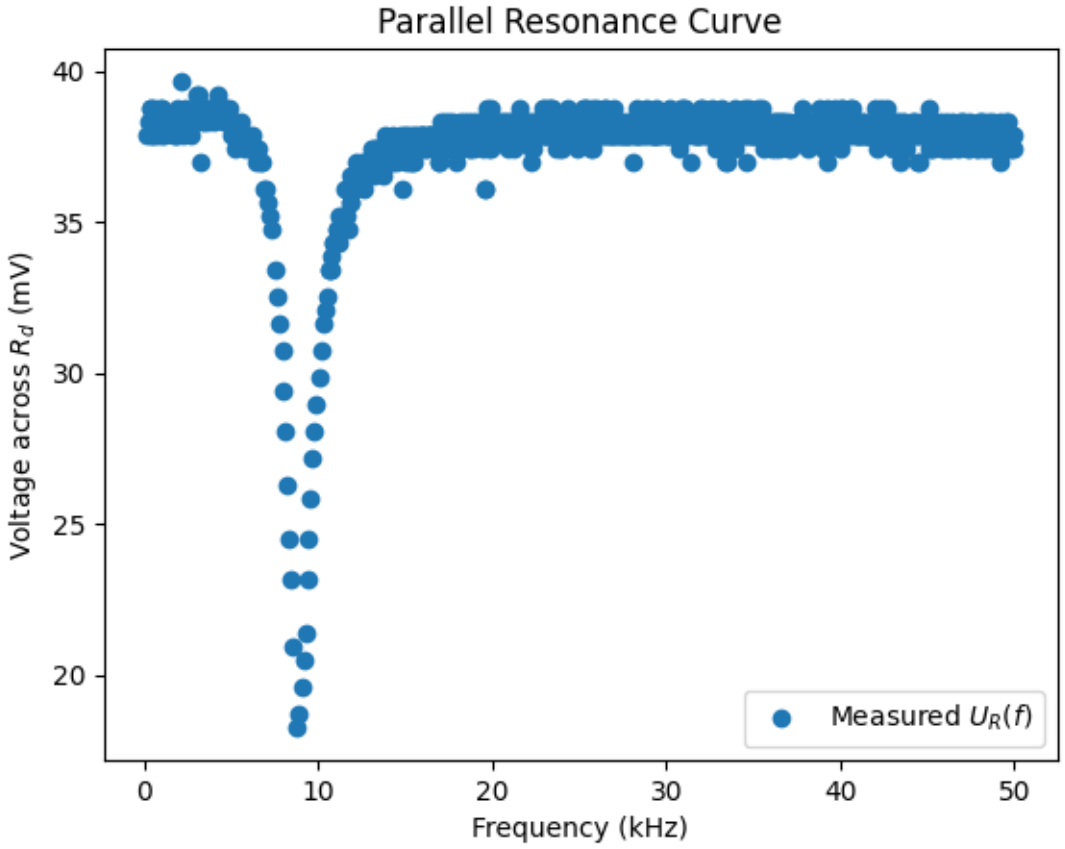
2 Task 3: Parallel Resonant Circuit

Objective: Measure the frequency-dependent current draw of the RLC circuit with capacitor and coil in parallel, series-connected to the damping resistor R_d.

2.1 Data Import and Preprocessing

[4]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Constants
r_d = 220          # Damping resistor in Ohm

# Load parallel dataset
data_p = pd.read_csv('Parallel_R1.csv')
freq_p = data_p.iloc[:, 2].astype(float)
A_p = data_p.iloc[:, 1].astype(float)

# Noise filtering threshold
mask = A_p > 0.01
freq_p = freq_p[mask]
A_p = A_p[mask]
```

```
# Plot raw resonance curve
plt.figure()
plt.plot(freq_p/1000, A_p*1000, 'o', label='Measured $U_R(f)$')
plt.xlabel('Frequency (kHz)')
plt.ylabel('Voltage across $R_d$ (mV)')
plt.title('Parallel Resonance Curve')
plt.legend()
plt.show()
```



3 Task 4: Evaluation of the Parallel Resonant Circuit

3.1 4a) Derivation of the Impedance Expression

For the circuit in Fig. 2, the coil (with series resistance R_{Sp} and inductance L) is in parallel with the capacitor C, and this combination is in series with the damping resistor R_d.

1. Impedance of coil: $Z_L = R_{Sp} + i\omega L$
2. Admittances in parallel: $Y_{LC} = \frac{1}{Z_L} + i\omega C = \frac{1}{R_{Sp}+i\omega L} + i\omega C$
3. Total admittance: $Y_{tot} = \frac{1}{R_d} + Y_{LC}$
4. Total impedance: $Z_{tot} = \frac{1}{Y_{tot}}$

Since we measure $U_R(f)$ across R_d, the current is $I(f) = U_R(f)/R_d$. Under constant source voltage U_0, we have:

$$I(f) = \frac{U_0}{|Z_{tot}(f)|} = \frac{U_0}{\left| \frac{1}{\frac{1}{R_d} + \frac{1}{R_{Sp}+i\omega L} + i\omega C} \right|}.$$

3.2 4b) Curve Fitting to Determine L, C, R_{Sp}, and R_d

[5]:
```
# Model function for I(f) under constant U0
def I_model(f, L, C, Rsp, Rd, U0):
    omega = 2 * np.pi * f
    Z_L = Rsp + 1j * omega * L
    Y_LC = 1.0 / Z_L + 1j * omega * C
    Y_tot = 1.0 / Rd + Y_LC
    Z_tot = 1.0 / Y_tot
    return U0 / np.abs(Z_tot)

# Initial guesses for parameters
p0 = [1.5e-3, 50e-9, 1.0, 220.0, max(A_p)]

# Perform curve fit
popt_p, pcov_p = curve_fit(
    I_model,
    freq_p,
    A_p,
    p0=p0,
    bounds=(
        [1e-4, 1e-11, 0.1, 10, 0],     # lower bounds
        [1e-2, 1e-6, 10, 1000, np.inf] # upper bounds
    )
)
L_fit, C_fit, Rsp_fit, Rd_fit, U0_fit = popt_p

print(f"Fitted parameters:\n  L = {L_fit:.3e} H\n  C = {C_fit:.3e} F\n  RSp =␣
 ↪{Rsp_fit:.1f} Ω\n  Rd = {Rd_fit:.1f} Ω\n  U0 = {U0_fit:.3f} V")
```

```
Fitted parameters:
  L = 1.000e-02 H
  C = 1.112e-07 F
  RSp = 0.1 Ω
  Rd = 10.0 Ω
  U0 = 0.366 V
```

3.3 4c) Resonance Frequency Comparison

The theoretical resonance frequency for the parallel circuit is given by Thomson's formula:

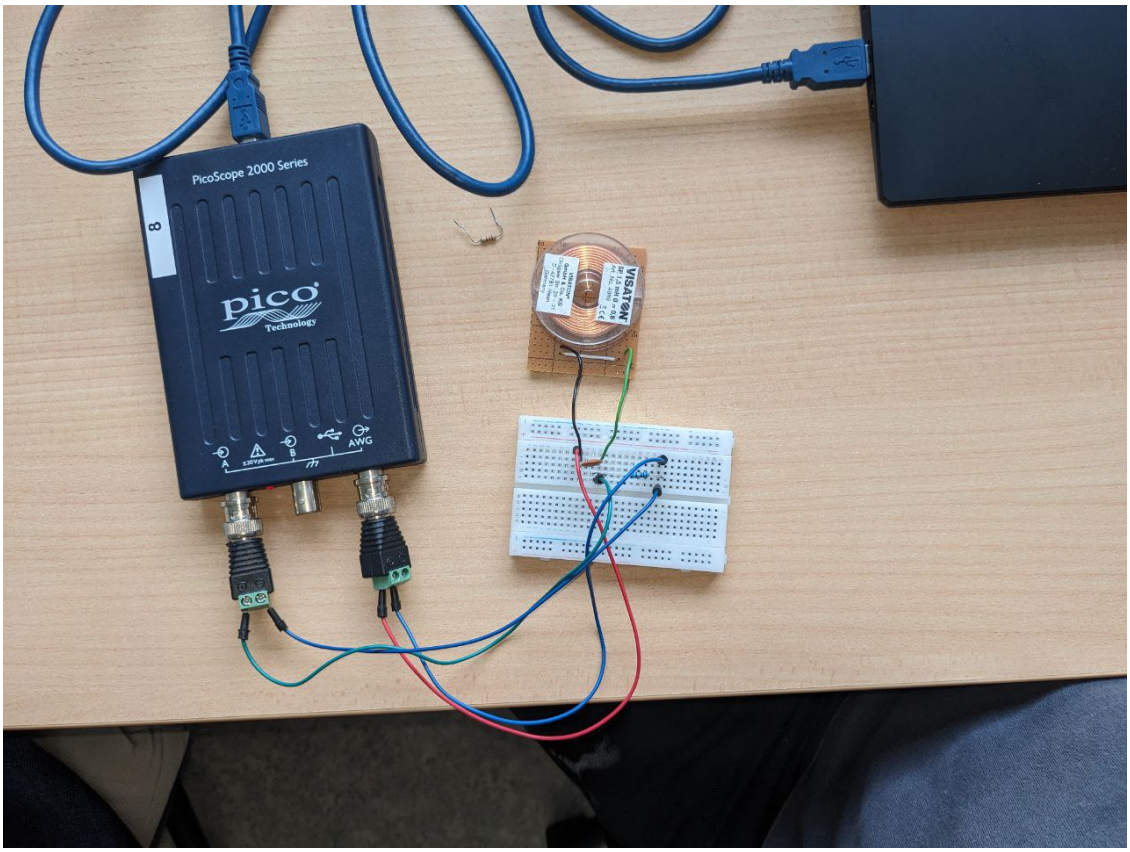
$$f_0 = \frac{1}{2\pi\sqrt{L_{fit}C_{fit}}}.$$

Compute and compare this with the series resonance frequency $f_{0,series}$ determined in Task 2:

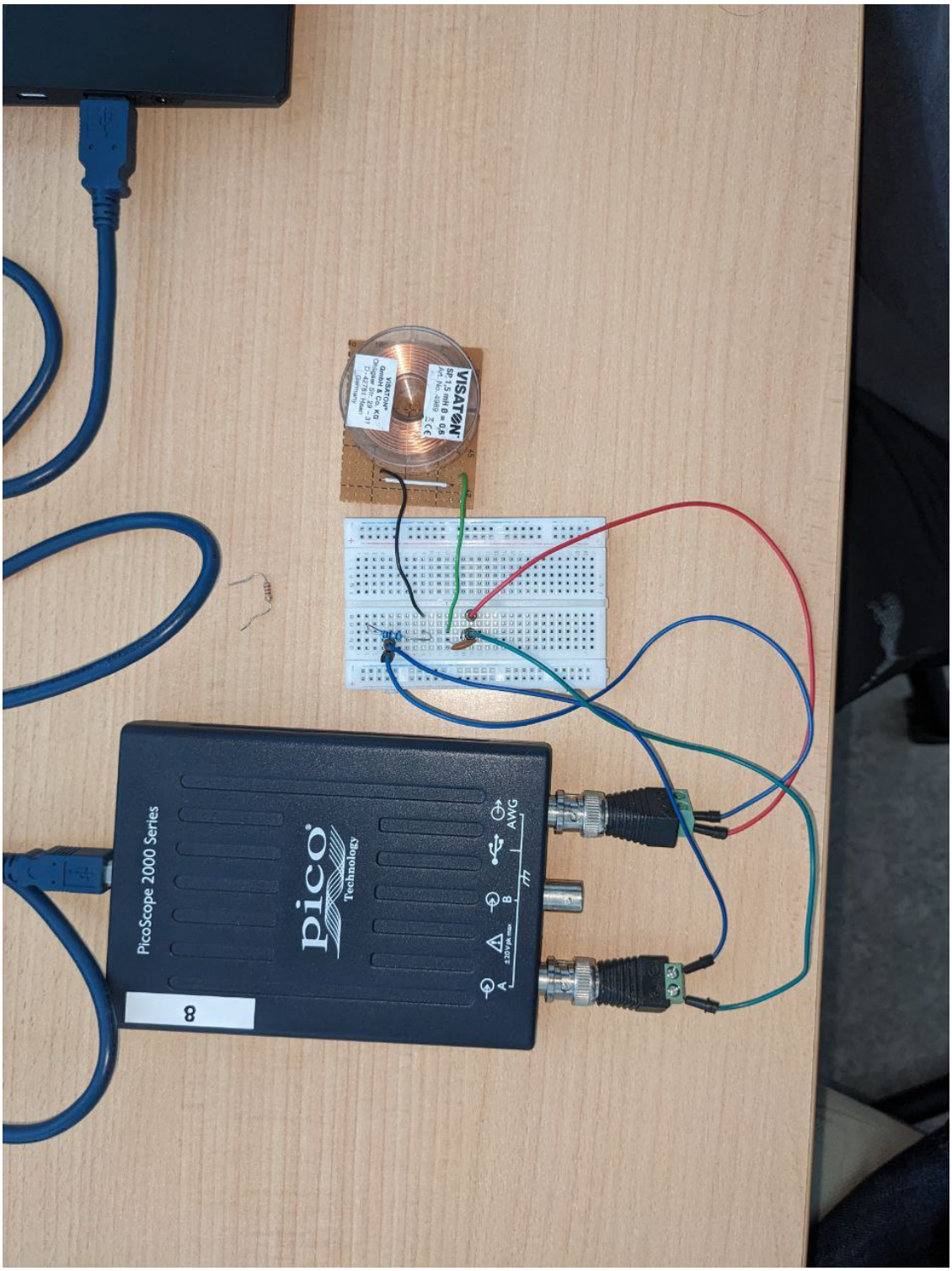
[7]:
```
f0_parallel = 1 / (2 * np.pi * np.sqrt(L_fit * C_fit))
print(f"Parallel f0 = {f0_parallel/1000:.2f} kHz")
print(f"Series   f0 = {f0/1000:.2f} kHz   (Task 2)")
```

```
Parallel f0 = 4.77 kHz
Series   f0 = 6.82 kHz   (Task 2)
```

3.4 Paralles Circuit



3.5 Series Circuit



E11e: Phase Shift in AC Circuits

Yaman Ajaj (3722952) ; Guilherme Schewtschik (3748052)

1. Setup:

The experiment investigates the phase shift between voltage and current in three different AC circuits: RLC, RC, and RL series circuits. The components include resistors, capacitors, and inductors, and the setup is powered by an arbitrary waveform generator. The voltage across the resistor (in-phase with current) and the total voltage are measured using a two-channel oscilloscope. We connected the AWG to the circuit and we kept on varying the frequency until we saw that the waves were in phase, meaning that is the resonance frequency. We then took measurements around that frequency and we recorded the time difference between the two signals. We did this for the three different circuits.

2. Theoretical Background:

n AC circuits, phase shifts occur due to the reactive components—capacitors and inductors. The impedance and corresponding phase angle (φ) for each circuit type are given by:

- Resistor (R): No phase shift.
- Capacitor (C): $Z_C = \frac{1}{j\omega C}$, causes current to lead the voltage $(\phi < 0)$.
- Inductor (L): $Z_L = j\omega L$, causes current to lag voltage $(\phi > 0)$.

For series circuits:

- RC Circuit: $\phi = -\tan^{-1}\left(\frac{1}{\omega R C}\right)$
- RL Circuit: $\phi = \tan^{-1}\left(\frac{\omega L}{R}\right)$
- RLC Circuit: $\phi = \tan^{-1}\left(\frac{\omega L - \frac{1}{\omega C}}{R}\right)$

The angular frequency is:

$$\omega = 2\pi f$$

The phase shift ϕ can be calculated from the time delay Δt using:

$$\phi = \omega \Delta t$$

Where:

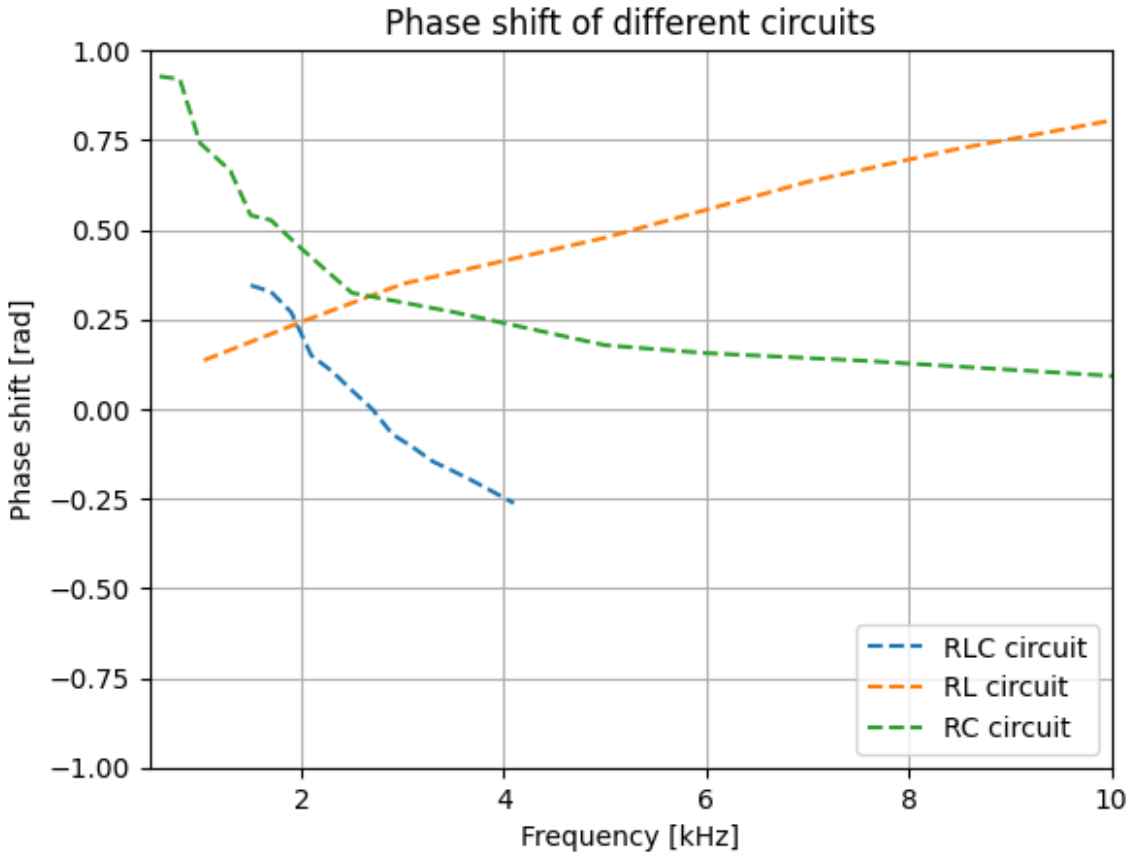
- f is the frequency in Hz.
- Δt is the time delay in seconds.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

#functions
def line(x, a, b):
    return a*x+b

#data import
data_1 = pd.read_excel('Data.xlsx', sheet_name='RLC')
data_2 = pd.read_excel('Data.xlsx', sheet_name='RL')
data_3 = pd.read_excel('Data.xlsx', sheet_name='RC')
data_4 = pd.read_excel('Data.xlsx', sheet_name='things')

data_RLC = data_1.to_numpy()
data_RL = data_2.to_numpy()
data_RC = data_3.to_numpy()
data_m = data_4.to_numpy()

#asserting variables
L , C , R_c, R_r = data_m[0,0]*1e-3, data_m[0,1]*1e-6, data_m[0,2],
data_m[0,3]
f_RLC, dt_RLC, rs_f = data_RLC[:,0]*1e3 , data_RLC[:,1]*1e-6,
data_RLC[0,3]*1e3
f_RL, dt_RL= data_RL[:,0]*1e3 , data_RL[:,1]*1e-6
f_RC, dt_RC = data_RC[:,0]*1e3 , data_RC[:,1]*1e-6


#calculating the phase shift and omega
omega_RC = 2*np.pi*f_RC
phi_RC = omega_RC*dt_RC

omega_RLC = 2*np.pi*f_RLC
phi_RLC = omega_RLC*dt_RLC

omega_RL = 2*np.pi*f_RL
phi_RL = omega_RL*dt_RL
```

Plotting the phase shift for all circuits:

```
plt.title('Phase shift of different circuits')
plt.xlim((0.5,10))
plt.ylim((-1,1))
plt.xlabel('Frequency [kHz]')
plt.ylabel('Phase shift [rad]')
```

```
plt.grid(True)

plt.plot(f_RLC*1e-3, phi_RLC, '--', label='RLC circuit')
plt.plot(f_RL*1e-3, phi_RL, '--', label='RL circuit')
plt.plot(f_RC*1e-3, phi_RC, '--', label='RC circuit')

plt.legend(loc = 'lower right')
plt.show()
```



Series Resistance of RC Circuit:

From the RC phase shift equation:

$$tan(|\phi|) = \frac{1}{\omega RC} \Longrightarrow R = \frac{1}{\omega C tan(|\phi|)}$$

The resistance of the resistor used n the RC circuit is:

$$R_r = 328.80\,\Omega$$

```
#resistance RC circuit
R_RC = np.mean(1/(np.tan(phi_RC)*omega_RC*C))
```

```
print(f'calculated resistance: R = {"%.3f"%(R_RC)} Ω')
print(f'error resistance: {"%.3f"%(((R_RC-R_r)/R_r)*1e2)}%')
```

```
calculated resistance: R = 330.231 Ω
error resistance: 0.435%
```

Inductance of the Coil:

i) From RLC Resonance

$$f_0 = \frac{1}{2\pi\sqrt{LC}} \Longrightarrow L = \frac{1}{4\pi^2 f_0^2 C}$$

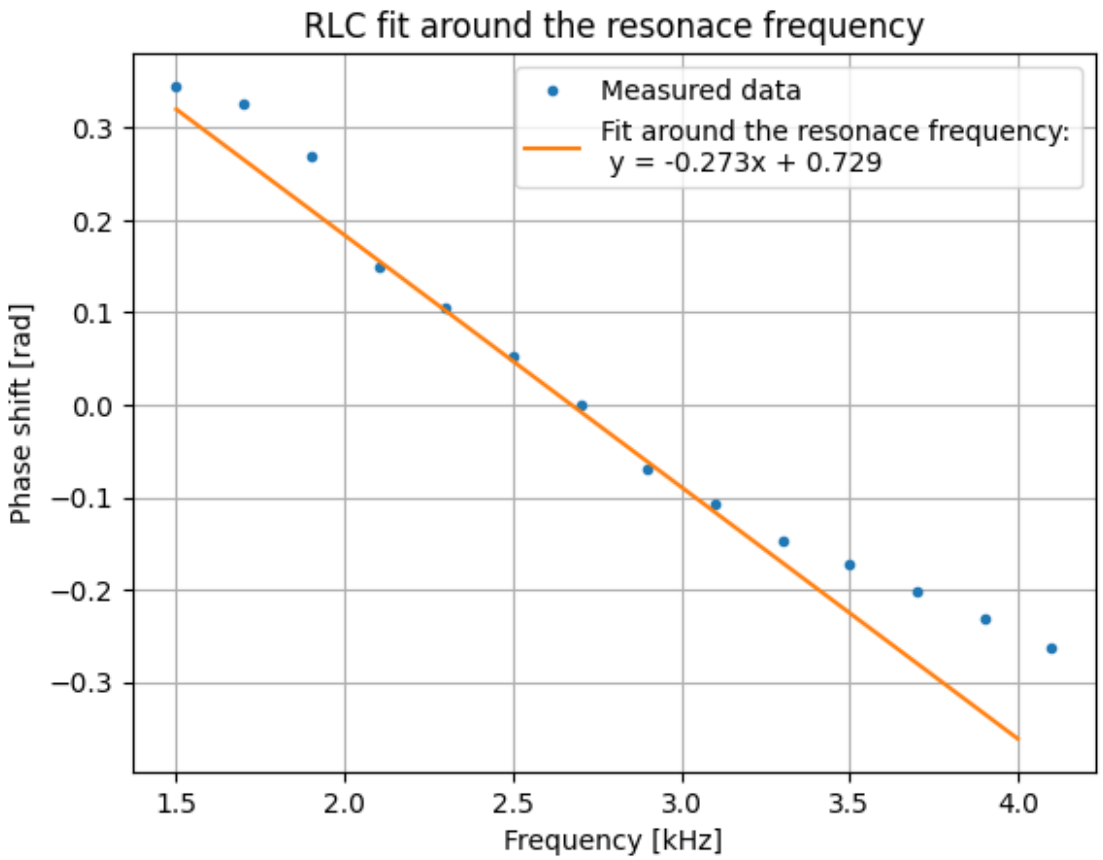
The resonance frequency measured experimentaly is:

$$f_0 = 2.68\,kHz$$

We can also calculate the resonance frequency by fitting a line to the phase shift around the experimental value:

```
#fitting a line around the resonance frequency
mask1 = (f_RLC < 3000)&(f_RLC > 2000)

p,c = curve_fit(line, f_RLC[mask1] , phi_RLC[mask1])

x = np.arange(min(f_RLC), max(f_RLC), 100)
fit1 = line(x, *p)

plt.figure(2)
plt.title('RLC fit around the resonace frequency')
plt.xlabel('Frequency [kHz]')
plt.ylabel('Phase shift [rad]')
plt.grid(True)

plt.plot(f_RLC*1e-3, phi_RLC, '.', label='Measured data')
plt.plot(x*1e-3, fit1, '-' , label = f'Fit around the resonace
frequency:\n y = {"%.3f"%(p[0]*1e3)}x + {"%.3f"%(p[1])}')

plt.legend(loc = 'upper right')
plt.show()
```

```
<matplotlib.legend.Legend at 0x24718f49420>
```




```
#fit data
rs_f_fit = -p[1]/p[0] #calculated resonance frequency

L_fit_RLC = 1/(C*((2*np.pi*rs_f_fit)**2))

print(f'Measured resonace frequency: {rs_f*1e-3} kHz, Fitted resonance
frequency: {"%.3f"%(rs_f_fit*1e-3)} kHz,\nMeasured Inductance =
{L*1e3} mH , Fitted Inductance = {"%.3f"%(L_fit_RLC*1e3)} mH')
print(f'error resonance frequency: {"%.3f"%(((rs_f_fit-
rs_f)/rs_f)*1e2)}%')
print(f'error inductance: {"%.3f"%(((L_fit_RLC-L)/L)*1e2)}%')
```

```
Measured resonace frequency: 2.68 kHz, Fitted resonance frequency:
2.673 kHz,
Measured Inductance = 6.77 mH , Fitted Inductance = 6.751 mH
error resonance frequency: -0.246%
error inductance: -0.285%
```

ii) From RL Phase fit:

$$\phi = \tan^{-1}\left(\frac{\omega L}{R}\right)$$

$$L = \frac{tan(\phi) R}{\omega}$$

```
L_fit_RL = np.mean(np.tan(phi_RL)*(R_r/omega_RL))

print(f'Fitted Inductance = {"%.3f"%(L_fit_RL*1e3)} mH')
print(f'error inductance: {"%.3f"%(((L_fit_RL-L)/L)*1e2)}%')
```

```
Fitted Inductance = 5.633 mH
error inductance: -16.795%
```

Fitting the Phase Shift of the RLC Circuit

We fit the RLC model using:

$$\phi = \tan^{-1}\left(\frac{\omega L - \frac{1}{\omega C}}{R}\right)$$

References:

- Tipler, Physics, 3rd Edition, Vol. 2, Sections 23-2, 28-2, 28-3, 28-5
- Demtröder, Electrodynamics and Optics, Sections 5.2, 5.4, 5.5

Experiment: E13He – Capacitor Discharge with MOSFET Switching

Yaman Ajaj (3722952) ; Guilherme Schewtschik (3748052)

Setup

In this experiment, we investigated the transient behavior of a capacitor discharging through a low-resistance path controlled by a MOSFET switch, using an Arduino Uno and Picoscope for control and measurement.

Components used:

- Arduino Uno (3.3 V logic)
- Maker-Factory IRF520 MOSFET switch module
- Capacitor
- Resistors
- Breadboard and jumper wires
- Picoscope 2204A with PicoScope 6 software

Circuit Description:

The circuit was built as follows:

- The capacitor is charged through a 1 kΩ resistor from Arduino's 3.3 V output.
- A MOSFET switch (IRF520) is connected such that its drain (V−) is wired to the negative terminal of the capacitor through a discharge resistor.
- Gate (Vin) is controlled by Arduino pin D2, and source (GND) is connected to common ground.
- The Picoscope measures the voltage across the capacitor to observe discharge behavior.

The Arduino was programmed to periodically switch the MOSFET on/off, allowing the capacitor to charge and discharge. The Picoscope captures the voltage across the capacitor during these cycles.

Circuit

ResCircuit

Measurement Technique:

The Picoscope was configured in:

- Single trigger mode
- Falling edge trigger on Channel A
- Appropriate time base (10 µs/div)
- Voltage range set to ± 10 V

Theoretical Background:

When a charged capacitor is discharged through a resistor, the voltage across it follows an exponential decay governed by the differential equation:

$$V(t)=V_0 e^{-\frac{t}{RC}}$$

where:

- $V(t)$ is the voltage across the capacitor at time t,
- V_0 is the initial voltage (3,3 V)
- R is the resistance in the discharge path (1 kΩ)
- C is the capacitance of the capacitor
- $\tau = RC$ is the time constant of the circuit.

Data analysis:

We performed measurements for three different Resistors values ($1\,K\Omega, 100\,\Omega, 22\,K\Omega, 100\,G\Omega$). The data was exported from PicoScope as CSV files and analyzed using Python.

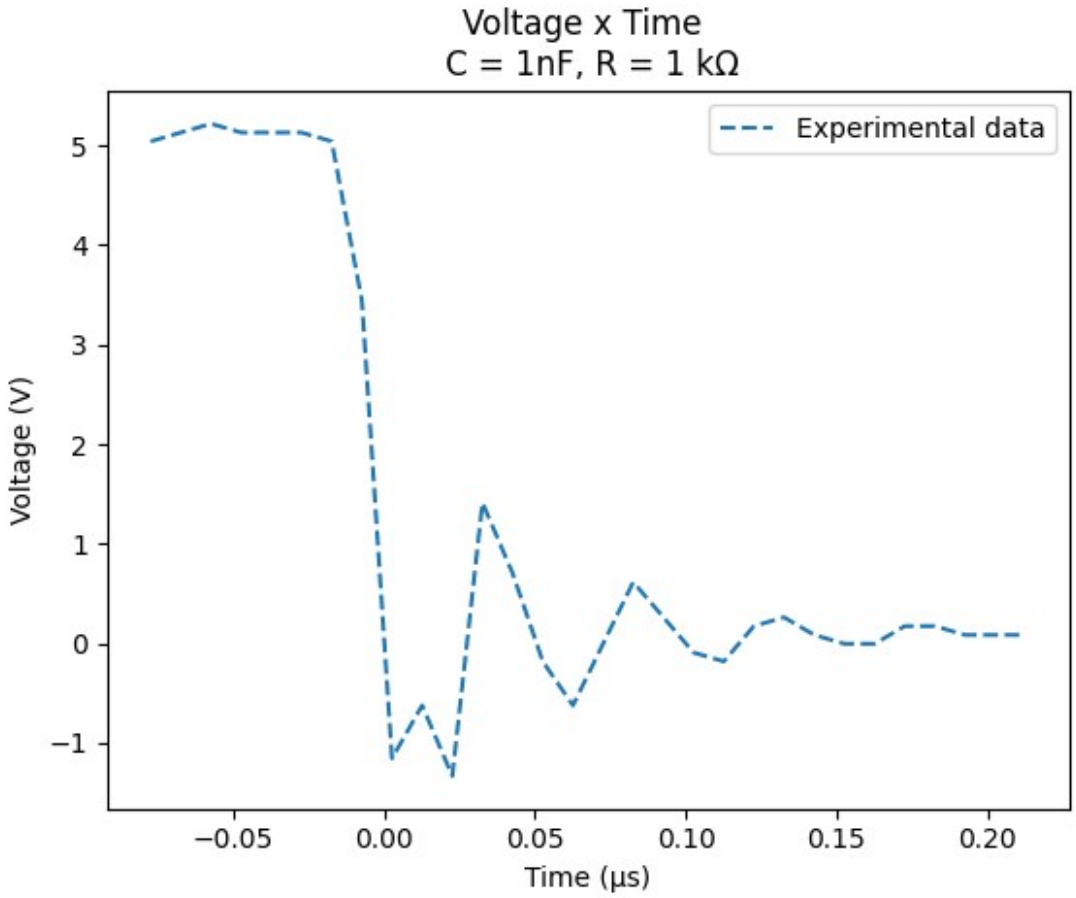
Task 1: Initial Measurement

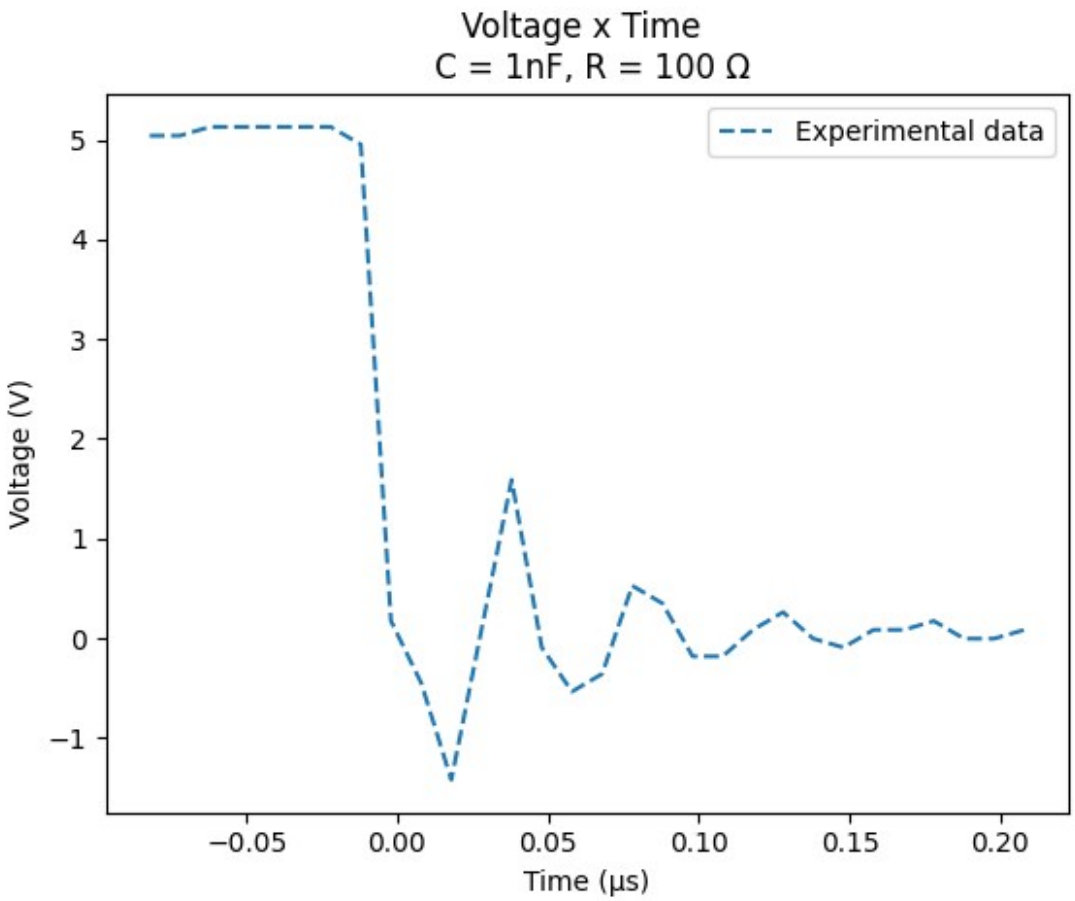
We performed a first measurement using a 1 nF capacitor, and then a 220 nF capacitor, discharged through a low resistance path via the MOSFET. The Picoscope recorded the voltage drop across the capacitor in “Single” trigger mode.

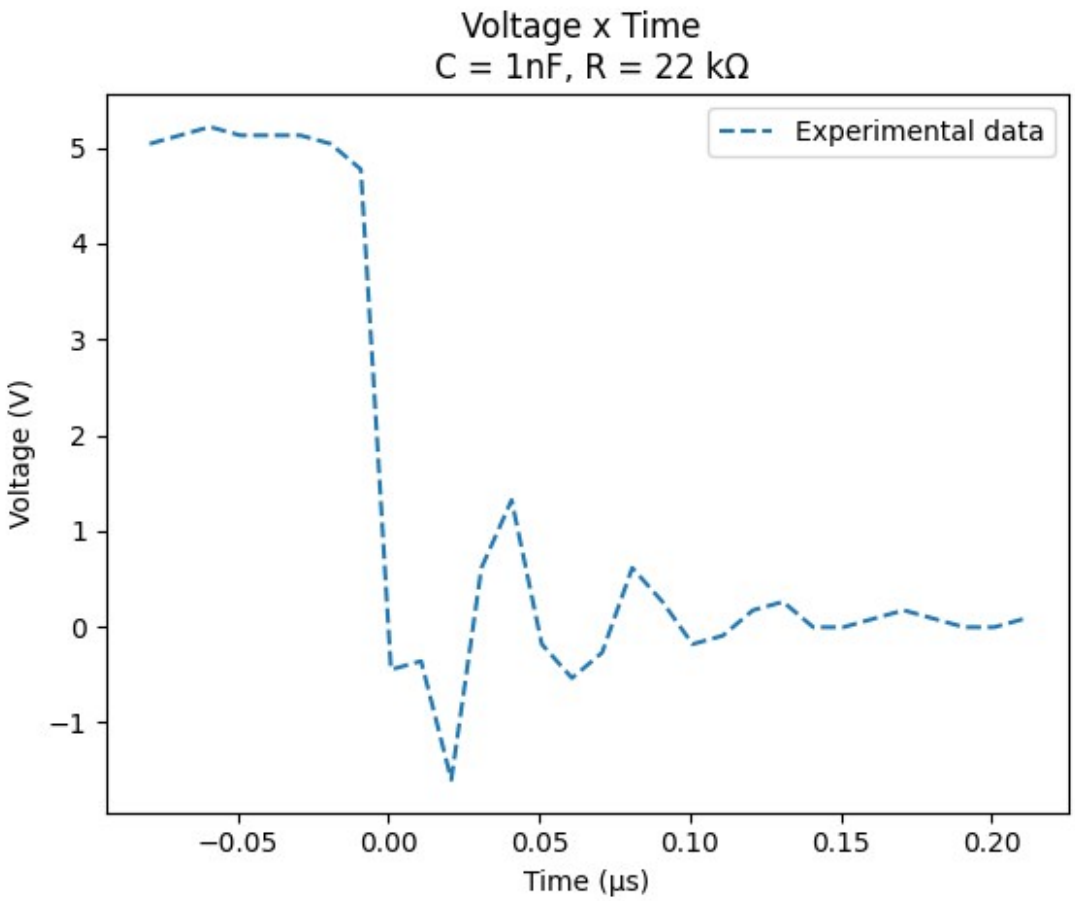
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

#description of each experiment
names = ['C = 1nF, R = 1 kΩ', 'C = 1nF, R = 100 Ω', 'C = 1nF, R = 22
kΩ','C = 1nF, R = 100 GΩ','C = 220nF, R = 1 kΩ']

#data import
data1 = pd.read_csv('wave1.csv', sep=';')
V1 = data1['(V)'].to_numpy()
```

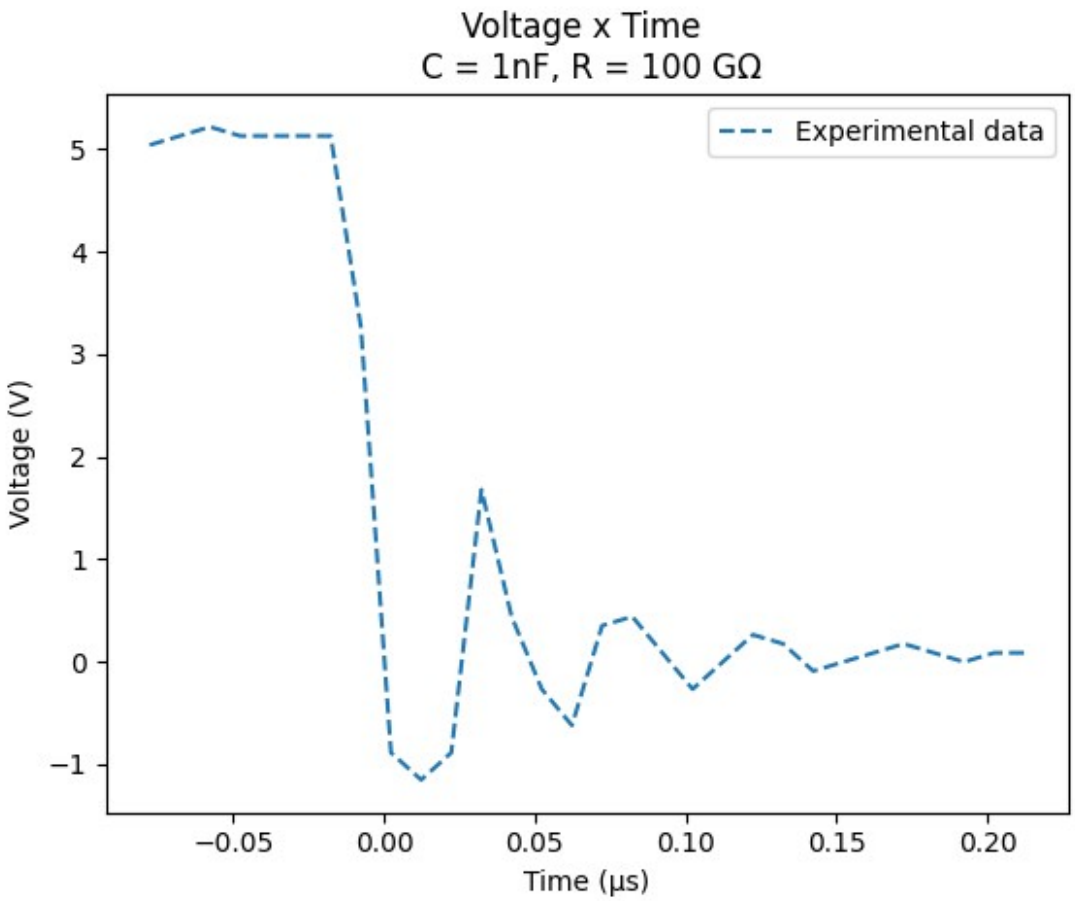
```
t1 = data1['(us)'].to_numpy()

data2 = pd.read_csv('wave2.csv', sep=';')
V2 = data2['(V)'].to_numpy()
t2 = data2['(us)'].to_numpy()

data3 = pd.read_csv('wave3.csv', sep=';')
V3 = data3['(V)'].to_numpy()
t3 = data3['(us)'].to_numpy()

data4 = pd.read_csv('wave4.csv', sep=';')
V4 = data4['(V)'].to_numpy()
t4 = data4['(us)'].to_numpy()

data5 = pd.read_csv('wave5.csv', sep=';')
V5 = data5['(V)'].to_numpy()
t5 = data5['(us)'].to_numpy()

#organizing the data
V = np.array([V1,V2,V3,V4,V5])
t = np.array([t1,t2,t3,t4,t5])

#difference between each element of V
dV = np.diff(V)

V_ind = np.array([])
t_ind = np.array([])

for i in range(0,5):
    #creating a mask aroun the voltage shift
    index_max_v = np.nonzero(dV[i] == max(dV[i]))[0]
    index_around_max = np.arange(index_max_v[0]-10, index_max_v[0]+20)

    V_ind = np.append(V_ind, [V[i][index_around_max]])
    t_ind = np.append(t_ind, [t[i][index_around_max]])

#reshaping the arrays
V_ind = V_ind.reshape(5,30)
t_ind = t_ind.reshape(5,30)

for i in range(0,5):
    plt.figure(f'Measurement {i+1}')
    plt.title(f'Voltage x Time \n {names[i]}')
    plt.xlabel('Time (µs)')
    plt.ylabel('Voltage (V)')

    plt.plot(t_ind[i], V_ind[i], '--', label = 'Experimental data')

    plt.legend(loc = 'upper right')
```

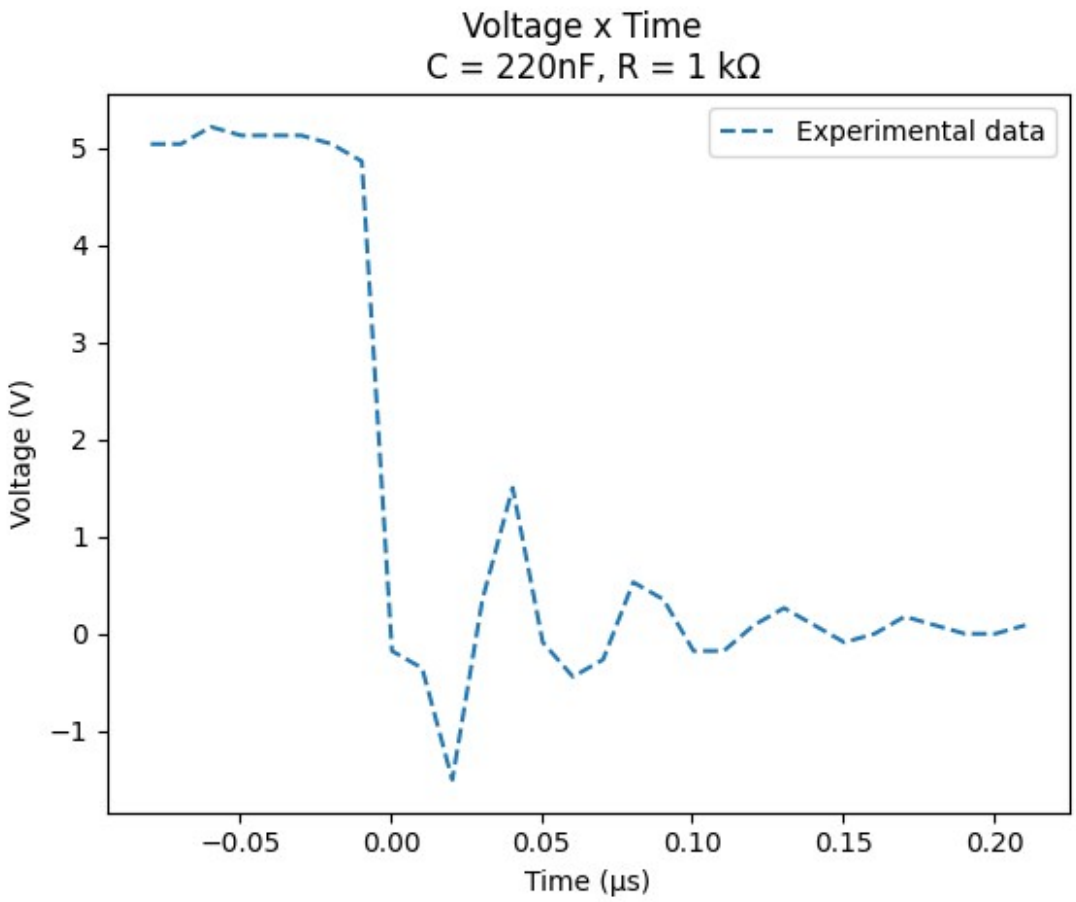
```
plt.show()
```


Voltage x Time
C = 1nF, R = 1 kΩ
Experimental data
Voltage (V)
5
4
3
2
1
0
−1
−0.05
0.00
0.05
0.10
0.15
0.20
Time (µs)


Voltage x Time
C = 1nF, R = 100 Ω
Experimental data
Voltage (V)
5
4
3
2
1
0
−1
−0.05
0.00
0.05
0.10
0.15
0.20
Time (μs)


Voltage x Time
C = 1nF, R = 22 kΩ
Experimental data
Voltage (V)
5
4
3
2
1
0
−1
−0.05
0.00
0.05
0.10
0.15
0.20
Time (μs)


Voltage x Time
C = 1nF, R = 100 GΩ
Experimental data
Voltage (V)
5
4
3
2
1
0
−1
−0.05
0.00
0.05
0.10
0.15
0.20
Time (μs)



Task 2: Model Development

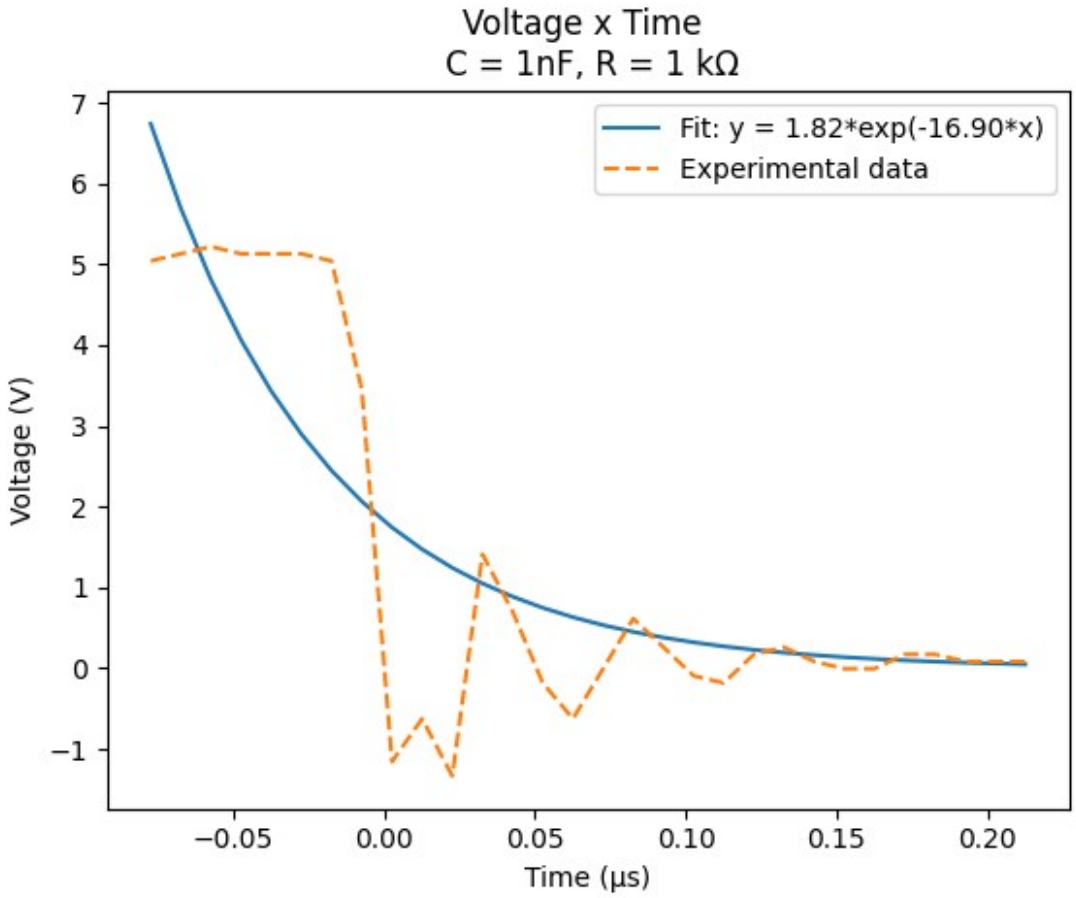
We developed a model based on the first-order linear differential equation:

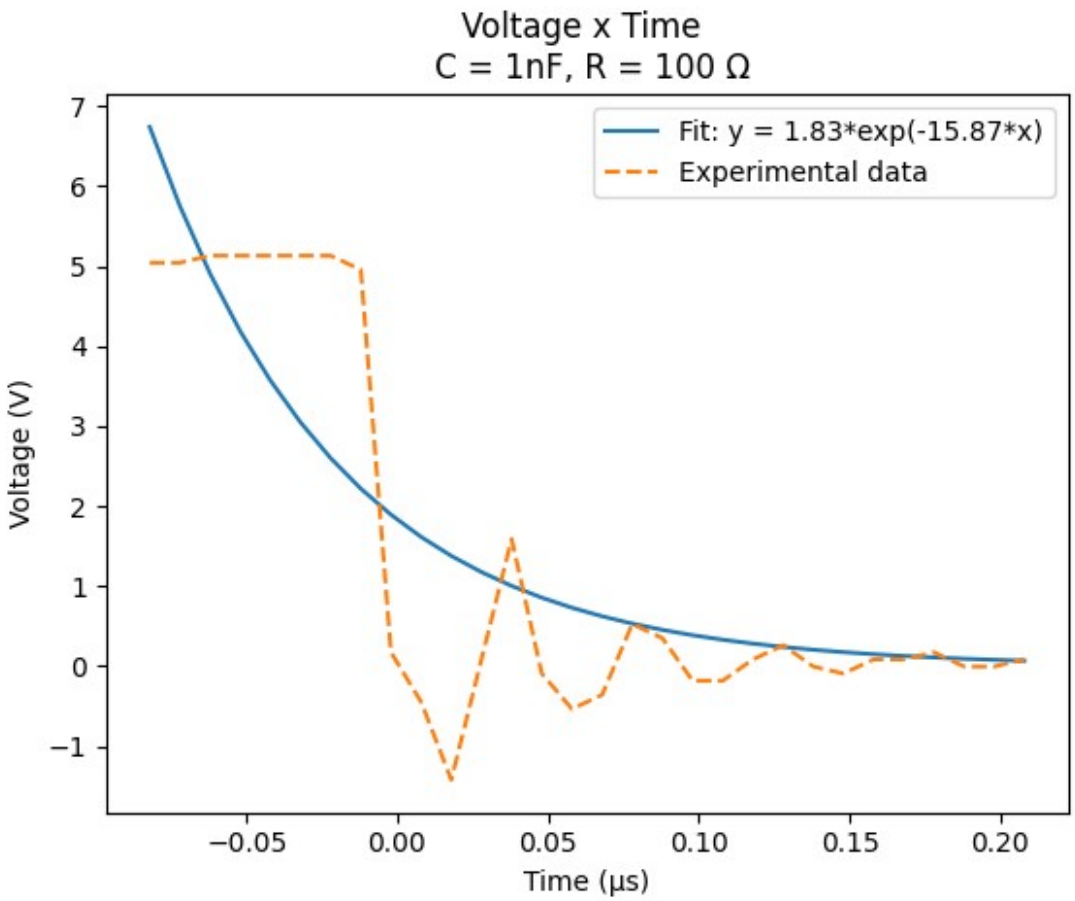
$$\frac{dV}{dt} = -\frac{1}{RC}V \rightarrow V(t) = V_0 e^{-\frac{t}{RC}}$$

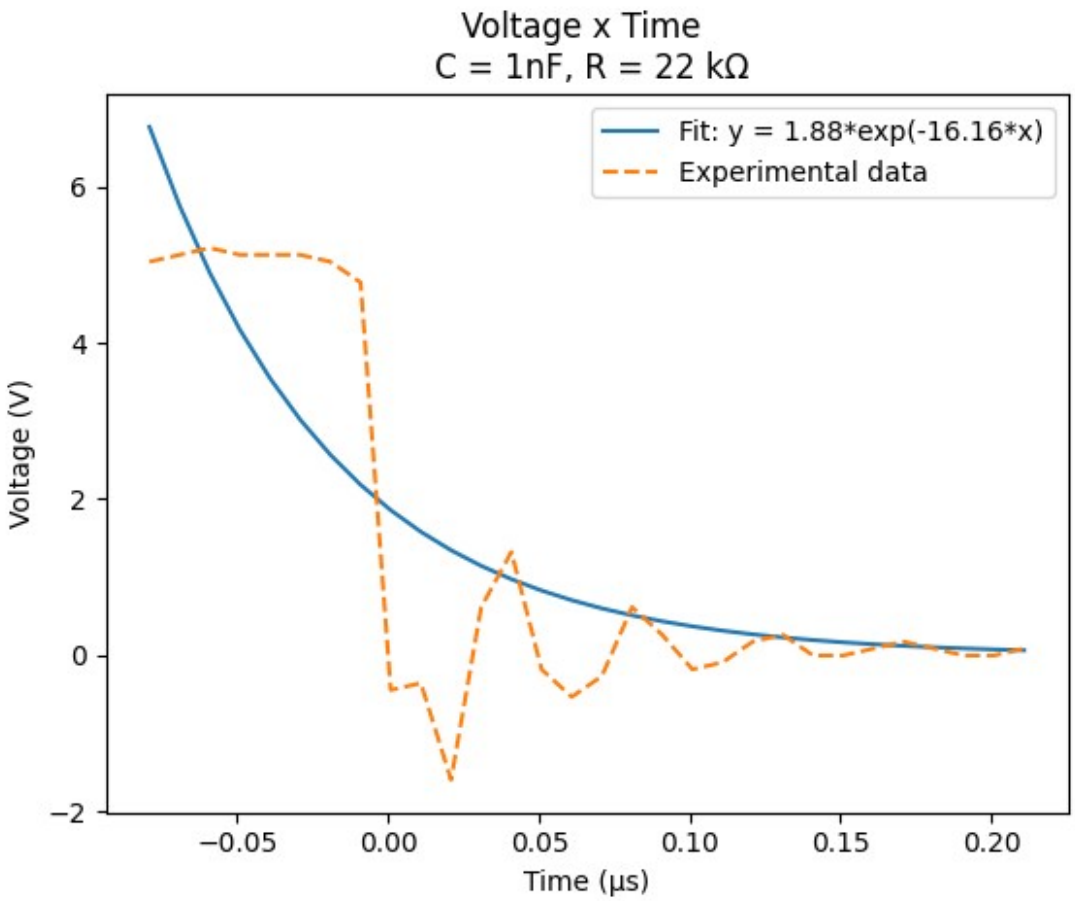
We validated this model by fitting the exponential function to our data using Python's curve_fit() method from scipy.optimize.

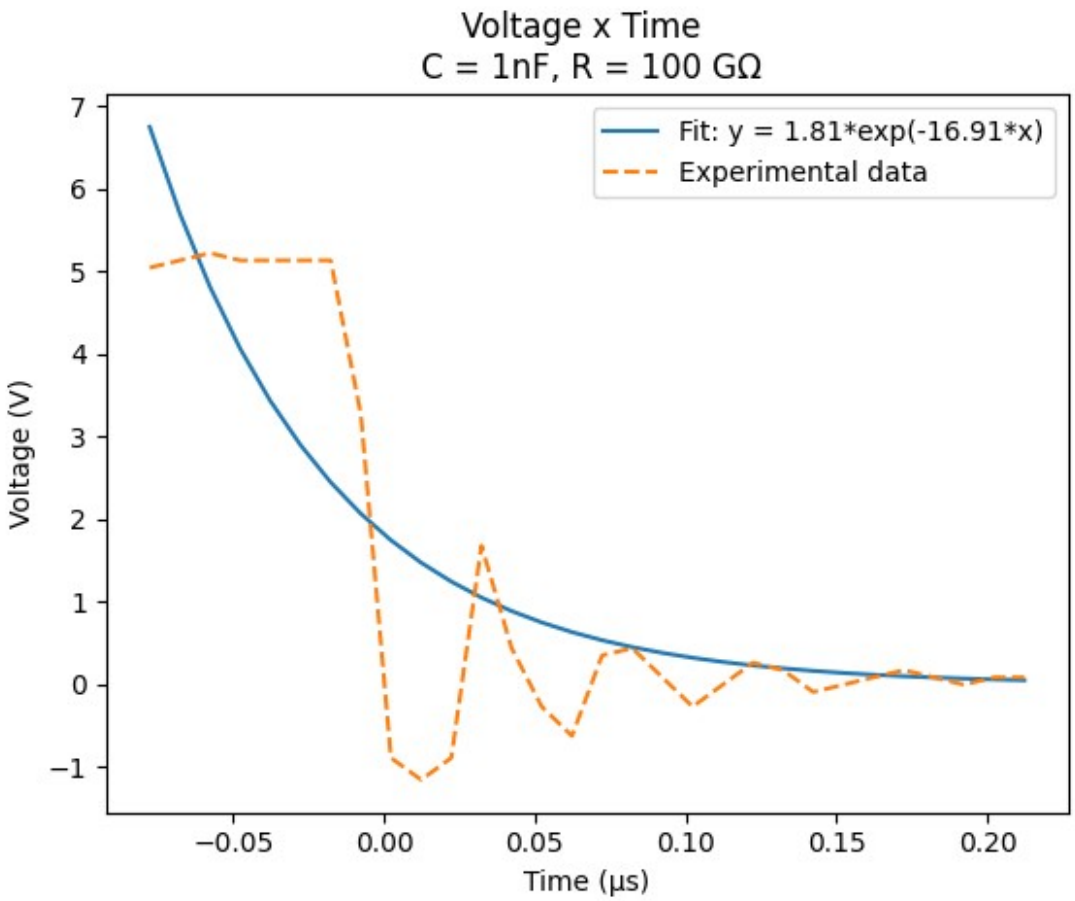
```
V_calc = np.array([])

popt = np.array([])

#curve for fitting
def exp(x,A,b):
    return A*np.exp(x*b)

#calculating the fit for each experiment
for i in range(0,5):
    x,_ = curve_fit(exp, t_ind[i] ,V_ind[i])
    popt = np.append(popt, x)
```

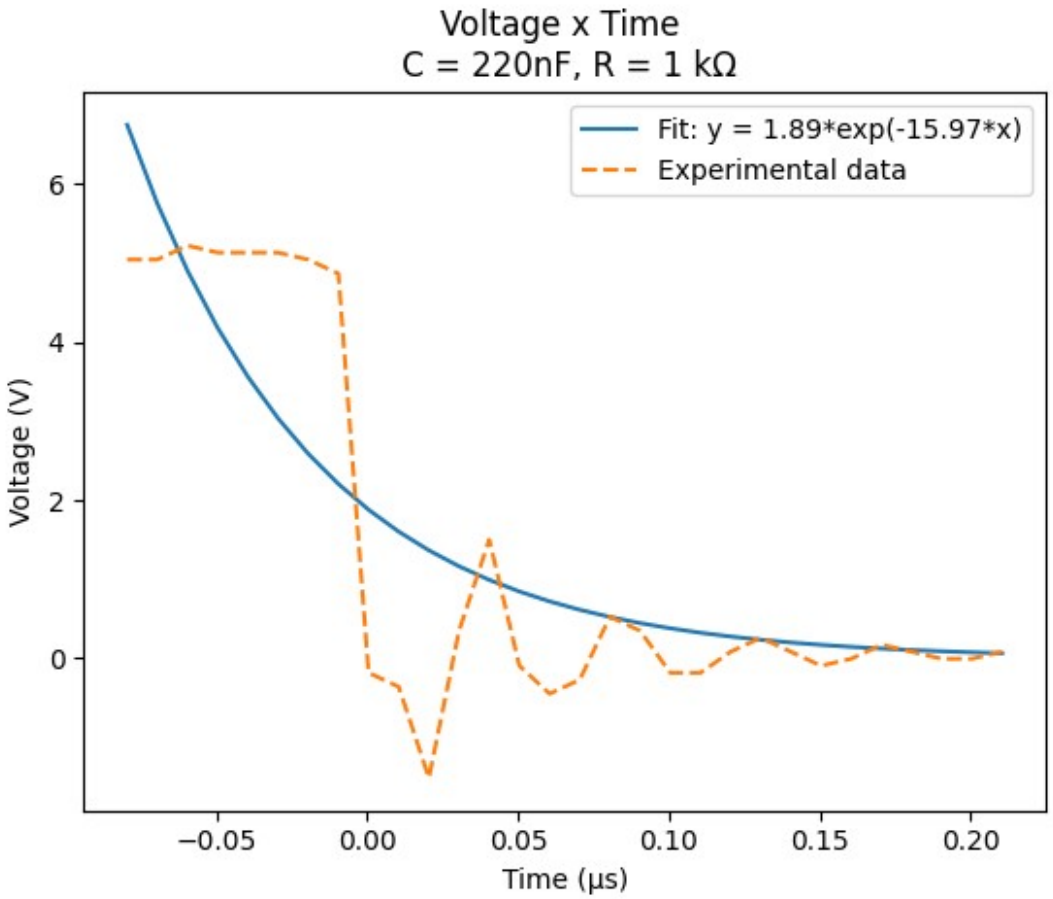
```
    V_calc = np.append(V_calc, exp(t_ind[i], x[0], x[1]))

#fit coeficients
popt = popt.reshape(5,2)

#reshaping the array
V_calc = V_calc.reshape(5,30)

#generating the graphs
for i in range(0,5):
    plt.figure(f'Fit {i+1}')
    plt.title(f'Voltage x Time \n {names[i]}')
    plt.xlabel('Time (µs)')
    plt.ylabel('Voltage (V)')

    plt.plot(t_ind[i], V_calc[i] , '-', label = f'Fit: y = {"%.2f"%
(popt[i,0])}*exp({"%.2f"%(popt[i,1])}*x)')
    plt.plot(t_ind[i], V_ind[i], '--', label = 'Experimental data')

    plt.legend(loc = 'upper right')

plt.show()
```




Voltage x Time
C = 1nF, R = 100 Ω
Fit: y = 1.83*exp(-15.87*x)
Experimental data
Voltage (V)
Time (µs)
7
6
5
4
3
2
1
0
−1
−0.05
0.00
0.05
0.10
0.15
0.20


Voltage x Time
C = 1nF, R = 22 kΩ
Fit: y = 1.88*exp(-16.16*x)
Experimental data
Voltage (V)
Time (µs)
6
4
2
0
−2
−0.05
0.00
0.05
0.10
0.15
0.20


Voltage x Time
C = 1nF, R = 100 GΩ
Fit: y = 1.81*exp(-16.91*x)
Experimental data
Voltage (V)
Time (µs)
7
6
5
4
3
2
1
0
−1
−0.05
0.00
0.05
0.10
0.15
0.20



Observations:

- All curves show a clear exponential decay.
- Fitted time constants agree with $\tau = RC$ within experimental error.
- This confirms the model's predictive power for varying capacitances and resistances.

E14

May 13, 2025

1 Lab Report: Semiconductor Diodes (E14e)

1.1 Yaman Ajaj (3722952) ; Guilherme Schewtschik (3748052)

1.2 1. Theoretical Background

Semiconductor diodes are key components in modern electronics. They are p-n junctions that exhibit nonlinear current-voltage (I–V) characteristics depending on the biasing condition.

1.2.1 1.1 Forward Bias Region

In forward bias, a diode allows current flow once the forward voltage exceeds the threshold voltage (typically 0.6–$0.7V$ for silicon, 2–$3V$ for LEDs). The current rises exponentially with voltage:

$$I = I_0(e^{\frac{qU}{nkT}} - 1)$$

Where: - I is the diode current. - U is the voltage across the diode. - q is the electron charge. - k is the Boltzmann constant. - n is the ideality factor. - T is the absolute temperature.

1.2.2 1.2 Reverse Bias Region

In reverse bias, a small saturation current flows until breakdown occurs (in Zener diodes), characterized by a sharp increase in reverse current at the breakdown voltage.

1.2.3 1.3 LED Emission

For LEDs, the voltage drop correlates with the photon energy $E = h\nu = qUE = h\nu = qU$, allowing estimation of the bandgap energy.

1.2.4 1.4 Junction Capacitance

In a reverse-biased p-n junction, the depletion region acts as a capacitor. The depletion capacitance C decreases with reverse voltage U, following:

$$\frac{1}{C^2} \propto U$$

This relation allows determination of the doping concentration from the slope of a $1/C^2$ vs U plot.

1.3 2. Experimental Setup

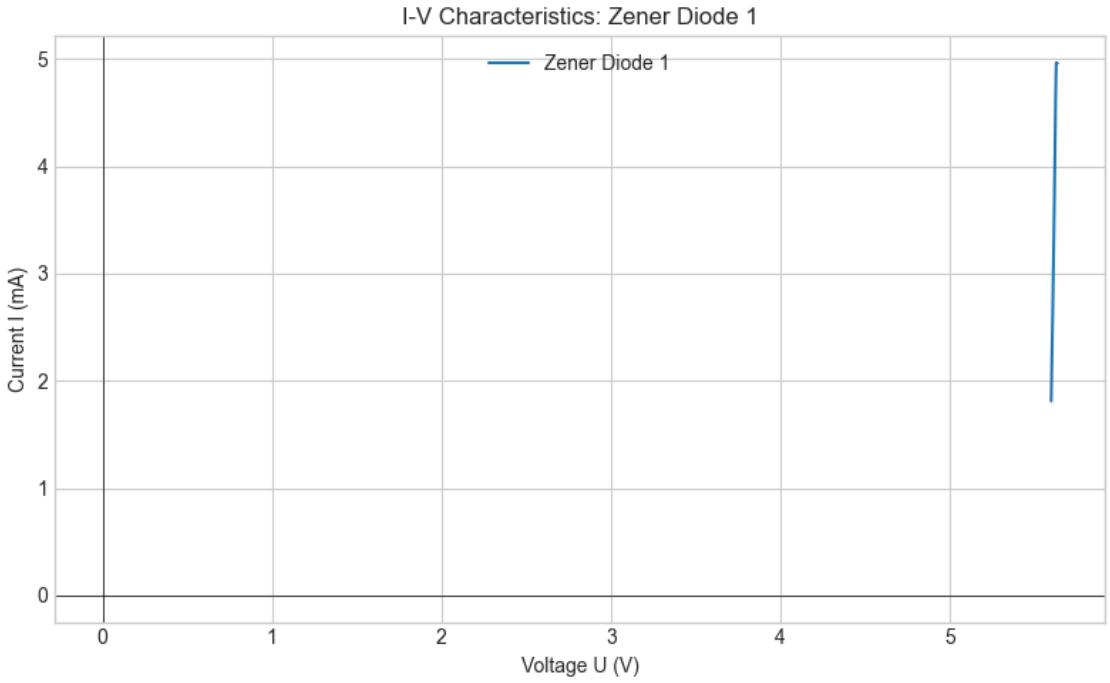
- Diodes tested: Silicon diode (Si), Light Emitting Diode (LED), Zener diode (Zener1 and Zener2).
- Measurements taken:
  - Current I vs Voltage U characteristics for forward and reverse bias
  - Capacitance vs reverse voltage using an LCR meter
- Equipment:
  - Source meter for I-V measurements
  - LCR meter for capacitance
  - Oscilloscope

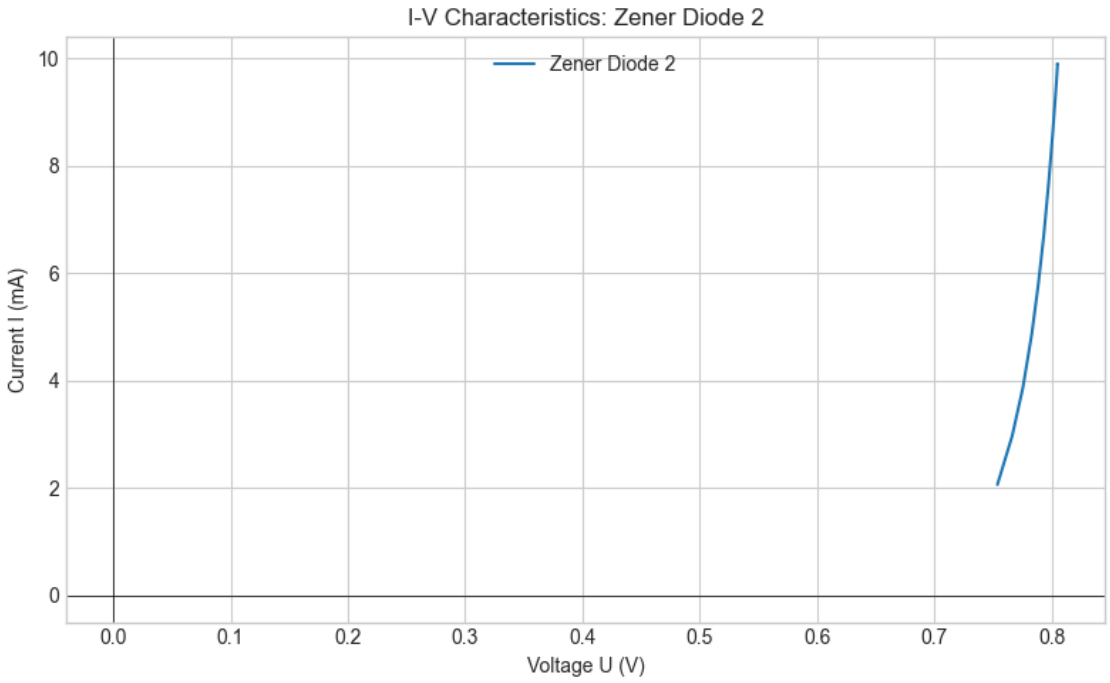
Data was logged digitally in CSV and Excel formats.

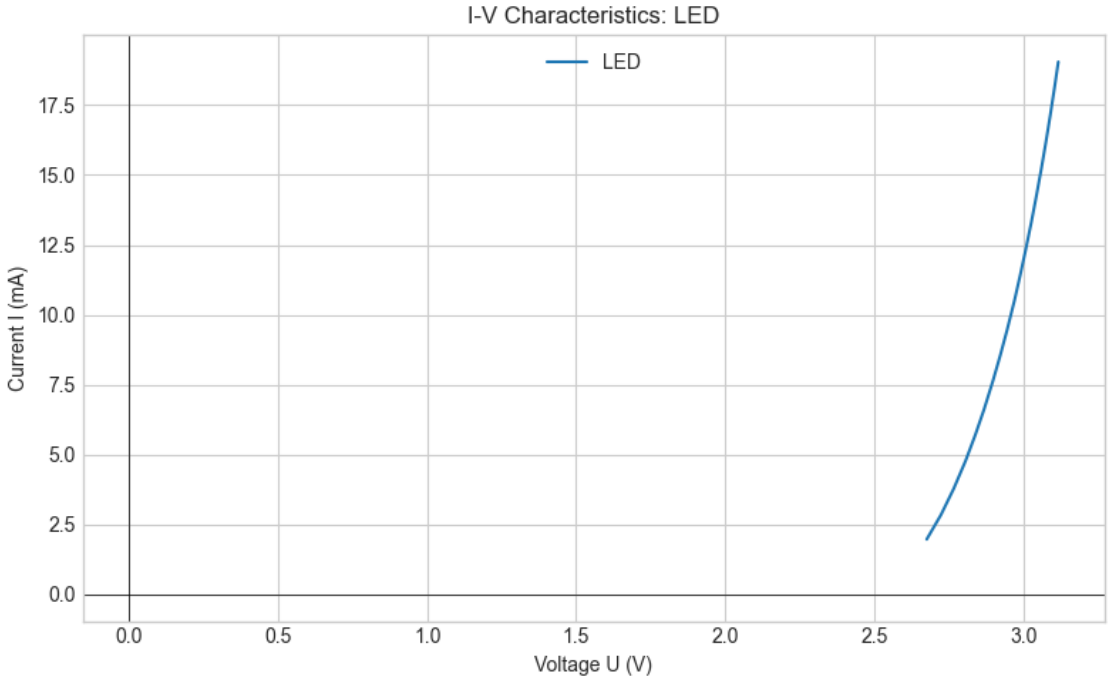
1.4 3. Data Analysis & Python Code

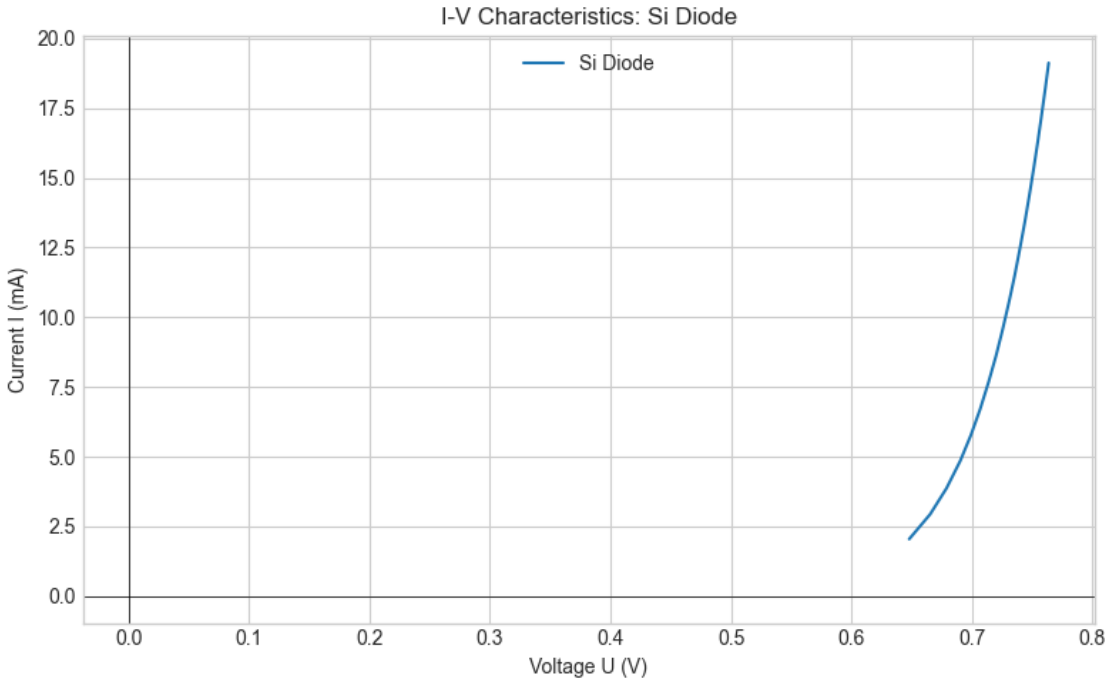
1.4.1 3.1 Task 1: I–V Characteristics (Zener1, Zener2, LED, Si)

Python code for loading and plotting:




I-V Characteristics: Zener Diode 2
Zener Diode 2
Current I (mA)
Voltage U (V)
0
2
4
6
8
10
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8


I-V Characteristics: LED
LED
Current I (mA)
Voltage U (V)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
0.0
0.5
1.0
1.5
2.0
2.5
3.0



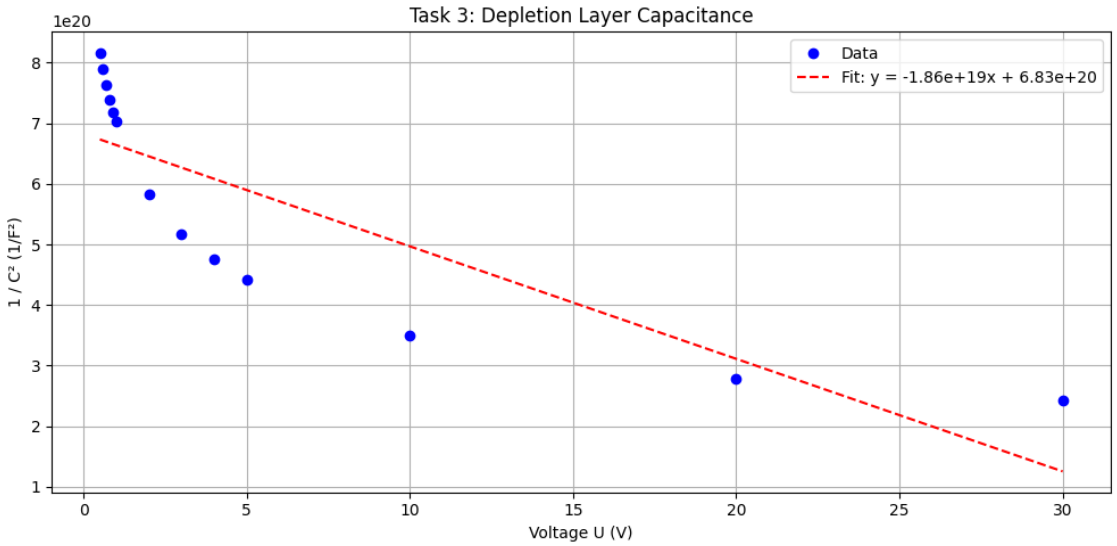
This produces a clear plot showing: - Exponential current increase for Si and LED in forward bias - Threshold voltages (~0.7V for Si, ~2.6–3.1V for LED) - Zener breakdown in reverse bias (around 5.6V for Zener1, ~0.8V for Zener2)

1.4.2 3.2 Task 2: LED Bandgap Estimation

We estimate the bandgap energy from the LED forward voltage $U_{on} \approx 2.6\text{–}3.1V$:

$$E_g \approx qU_{on} \rightarrow E_g \approx (1.602 \times 10^{-19}C) \times 3.0V = 3.0eV$$

1.4.3 3.3 Task 3: Depletion Layer Capacitance




```
Linear regression results:
Slope     = -1.857e+19  1/F²·V
Intercept = 6.825e+20  1/F²
R²        = 0.7008
```

E17e Fourier Analysis of Coupled Electric Oscillations

Yaman Ajaj (3722952); Guilherme Schewtschik (3748052)

Theoretical Background:

In this experiment, we investigate coupled electrical oscillations in LC-circuits using both capacitive and inductive coupling. The key theoretical concepts and governing equations are summarized below:

1. **Uncoupled Resonant Circuit** A single LC-resonator of inductance L and capacitance C exhibits free (undriven) oscillations whose angular frequency is:

$$\omega_0 = \frac{1}{\sqrt{LC}} \rightarrow f_0 = \frac{\omega_0}{2\pi}$$

   These arise from the balance of inductive and capacitive voltages under negligible damping

2. **Capacitive Coupling** Two identical LC-circuits coupled via a capacitor C_K can be arranged in two configurations:
   - **Low-point circuit**: The equations of motion for currents I_A, I_B in each loop are

$$\frac{d^2 I_A}{dt^2} + \frac{1}{LC} I_A + \frac{1}{LC_K}\left(I_A - I_B\right) = 0$$

$$\frac{d^2 I_B}{dt^2} + \frac{1}{LC} I_B + \frac{1}{LC_K}\left(I_B - I_A\right) = 0$$

   By forming $I_{+¿} = I_A + I_B ¿$ and $I_- = I_A - I_B$, one finds two eigenmodes:

- **In-phase** ($I_- = 0$) at frequency:

$$\omega_1 = \omega_0 = \sqrt{\frac{1}{LC}}$$

- **Out-of-phase** ($I_{+¿=0¿}$) at frequency:

$$\omega_2 = \frac{1}{\sqrt{L\left(C^{-1} + 2\,C_K^{-1}\right)^{-1}}}$$

  The capacitive coupling factor follows

$$k_{C,T} = \frac{C}{C + C_K}$$

- **High-point circuit** Charging the two capacitors either in-phase ($U_a = U_b$) or anti-phase ($U_a = -U_b$) excites the in- and out-of-phase modes directly. One obtains:

$$\omega_- = \sqrt{\frac{1}{LC}}$$

$$\omega_{+¿}=\frac{1}{\sqrt{L(C+2C_K)}} \rightarrow k_{C,H}=\frac{C_K}{C+C_k}¿$$

1. **Beat Oscillations** If only one circuit is excited ($I_A(0)\neq 0, I_B=0$), the superposition of the two modes leads to beats. The beat angular frequency is:

$$\omega_S=\omega_2-\omega_1 \rightarrow T_S=\frac{2\pi}{\omega_S}$$

1. **Inductive Coupling** Two identical LC-circuits coupled by mutual inductance M obey

$$\frac{d^2 I_A}{dt^2}L+M\frac{d^2 I_B}{dt^2}+\frac{1}{C}I_A=0$$
$$\frac{d^2 I_B}{dt^2}L+M\frac{d^2 I_A}{dt^2}+\frac{1}{C}I_B=0$$

Defining $k_L=\frac{M}{L}$, the eigenfunctions are:

$$\omega_{\pm}=\frac{\omega_0}{\sqrt{1\pm k_L}}$$

and for weak coupling $\Delta\omega=\omega_- - \omega_{+¿\approx k_L \omega_0¿}$.

Experimental Setup and Analysis:

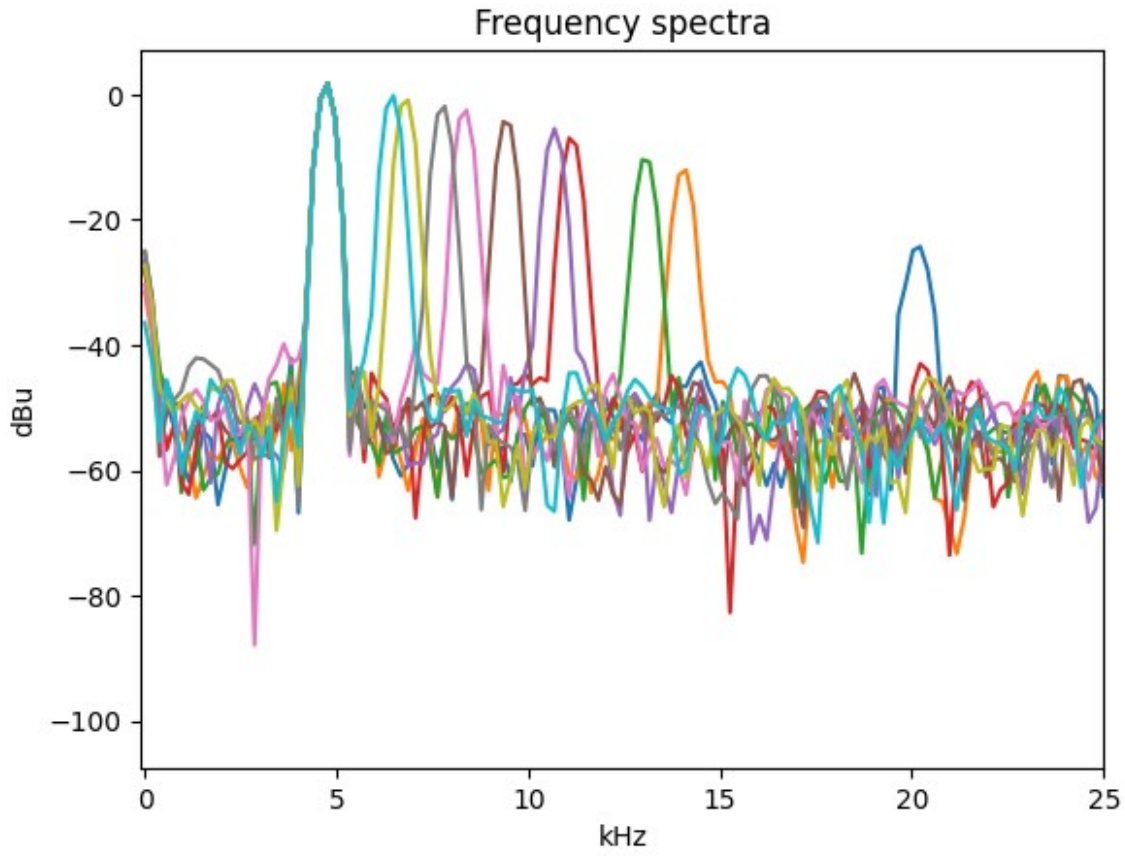
The experiment employs a USB-oscilloscope with FFT capability (PicoScope 2000/5000 series), variable capacitors, coils, resistors, and switches mounted on a circuit board. All measurements are recorded digitally and transferred to a PC for analysis

1. **Low-Point Capacitive Coupling**
   - Assemble the circuit as shown in the figure: two series LC branches in parallel with a variable C_K.
   - Charge capacitors via the DC source U_a, then close the switch to initiate free oscillations.
   - Record time-domain traces and use the FFT function to obtain frequency spectra
   - Repeat for ten values of C_K, exporting all ten spectra for a combined plot.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

#Values for Ck
Ck =
np.array([0.002,0.005,0.006,0.009,0.010,0.015,0.020,0.025,0.04,0.05])
Ck = 1.111e-2*Ck

#Data import
A_list = []
for i in range(1,11):
    data1 = pd.read_csv(f'Data/Task1/Task1_freq_{i}.csv', sep=',')
```

```
    freq = data1['Frequency'].to_numpy()
    y = data1['Channel A'].to_numpy()
    A_list.append(y)
A = np.vstack(A_list)

#Filtering the in and out frequencies
rfreq = []
for i in range(0,10):
    x = A[i]>-40
    f = freq[x]
    split = np.where(np.diff(f)>1)[0]
    segments = np.split(f, split+1)
    for j in range(0,len(segments)):
        a = np.mean(segments[j])
        rfreq.append(a)

#in the last measurament the in and out frequencies where so close it
couldnt be differentiated by a simple filter
rfreq.append(rfreq[-1])

rfreq = np.reshape(rfreq,(int(len(rfreq)/3),3))

f_in = rfreq[:,1]
f_out = rfreq[:,2]

#calculations
k = (f_out**2-f_in**2)/(f_out**2+f_in**2)
C = k*Ck/(1-k)

#results for k
print("The valued obtained for k given diferent Ck's is:")
for i in range(0,len(C)):
    print(f'k = {"%.2f"%(k[i])} for Ck = {"%.2f"%(Ck[i]*1e6)}µF')

#results for C
print("The valued obtained for C given diferent Ck's is:")
for i in range(0,len(C)):
    print(f'C = {"%.2f"%(C[i]*1e6)}µF for Ck = {"%.2f"%
(Ck[i]*1e6)}µF')

#plot
plt.figure(1)
for i in range(0,10):
    plt.title('Frequency spectra')
    plt.xlabel('kHz')
    plt.ylabel('dBu')
    plt.xlim(-0.1,25)
    plt.plot(freq, A[i])

plt.show()
```

```
The valued obtained for k given diferent Ck's is:
k = 0.90 for Ck = 22.22µF
k = 0.80 for Ck = 55.55µF
k = 0.77 for Ck = 66.66µF
k = 0.70 for Ck = 99.99µF
k = 0.68 for Ck = 111.10µF
k = 0.61 for Ck = 166.65µF
k = 0.53 for Ck = 222.20µF
k = 0.46 for Ck = 277.75µF
k = 0.35 for Ck = 444.40µF
k = 0.00 for Ck = 555.50µF
The valued obtained for C given diferent Ck's is:
C = 194.90µF for Ck = 22.22µF
C = 222.20µF for Ck = 55.55µF
C = 227.22µF for Ck = 66.66µF
C = 235.05µF for Ck = 99.99µF
C = 234.67µF for Ck = 111.10µF
C = 256.81µF for Ck = 166.65µF
C = 246.94µF for Ck = 222.20µF
C = 240.62µF for Ck = 277.75µF
C = 244.32µF for Ck = 444.40µF
C = 0.00µF for Ck = 555.50µF
```

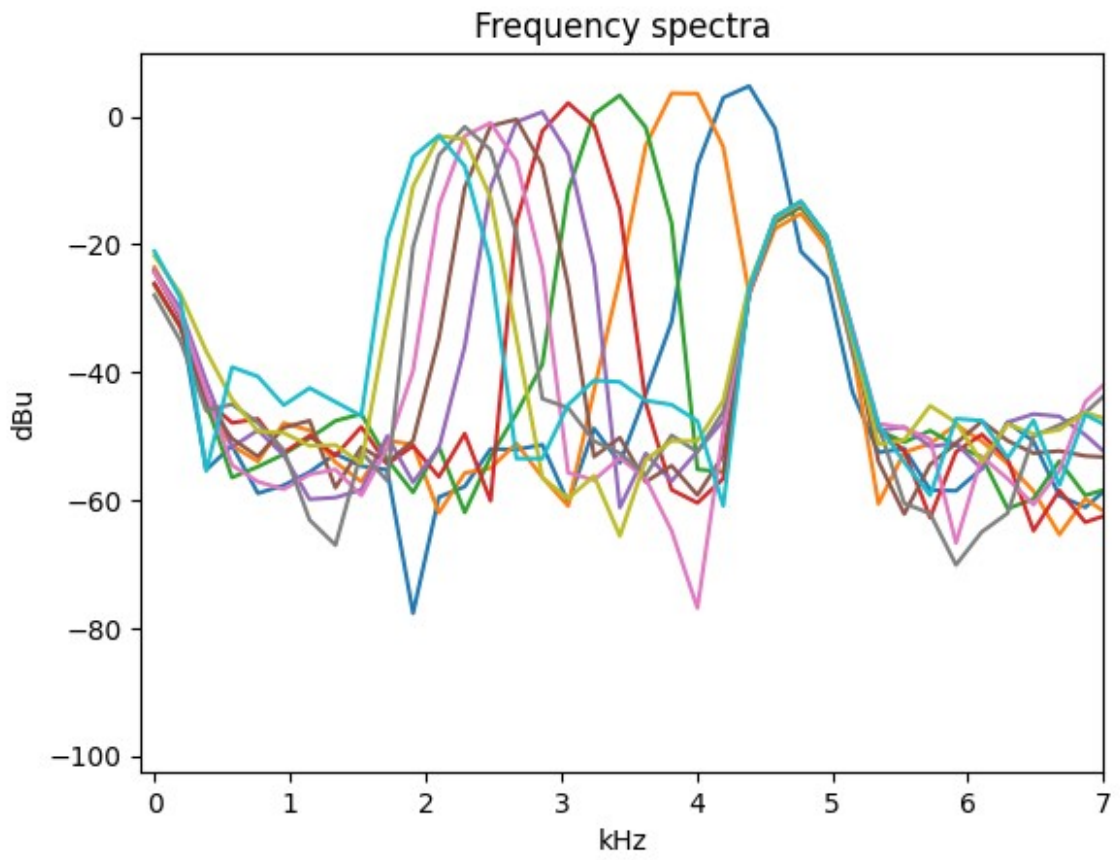


1. **High-Point Capacitive Coupling**
   - Build the high-point circuit as shown in the figure with separate charging switches S_a, S_b.
   - For in-phase mode: set $U_a = U_b$; for out-of-phase: $U_a = -U_b$.
   - Measure and record frequency spectra for ten values of C_K.

```
#Ck values
Ck =
np.array([0.002,0.005,0.01,0.015,0.02,0.025,0.03,0.035,0.04,0.045])

Ck = 1.111e-2*Ck

#data import
A_list = []
for i in range(1,11):
    data1 = pd.read_csv(f'Data/Task2/Task2_freq_{i}.csv', sep=',')
    freq = data1['Frequency'].to_numpy()
    y = data1['Channel A'].to_numpy()
    A_list.append(y)
A = np.vstack(A_list)

#filtering out other frequencies
rfreq = []
for i in range(0,10):
    x = A[i]>-40
    f = freq[x]
    split = np.where(np.diff(f)>0.5)[0]
    segments = np.split(f, split+1)
    for j in range(0,len(segments)):
        a = np.mean(segments[j])
        rfreq.append(a)

#adding noise to the frequencies that where too close together to
differentiate
rfreq = np.insert(rfreq, 2, rfreq[1]+np.random.rand()*1e-3)
rfreq = np.insert(rfreq, 4, rfreq[4]+np.random.rand()*1e-3)

rfreq = np.reshape(rfreq,(int(len(rfreq)/3),3))

#Calculating k and C
f_in = rfreq[:,1]
f_out = rfreq[:,2]

k = (f_out**2-f_in**2)/(f_out**2+f_in**2)

C = (1-k)*Ck/k

#results for k
print("The valued obtained for k given diferent Ck's is:")
for i in range(0,len(C)):
```

```
    print(f'k = {"%.2f"%(k[i])} for Ck = {"%.2f"%(Ck[i]*1e6)}µF')

#results for C
print("The valued obtained for C given diferent Ck's is:")
for i in range(0,len(C)):
    print(f'C = {"%.2f"%(C[i]*1e6)}µF for Ck = {"%.2f"%
(Ck[i]*1e6)}µF')

#plotting
plt.figure(1)
for i in range(0,10):
    plt.title('Frequency spectra')
    plt.xlabel('kHz')
    plt.ylabel('dBu')
    plt.xlim(-0.1,7)
    plt.plot(freq, A[i])

plt.show()
```

```
The valued obtained for k given diferent Ck's is:
k = 0.00 for Ck = 22.22µF
k = -0.00 for Ck = 55.55µF
k = 0.34 for Ck = 111.10µF
k = 0.42 for Ck = 166.65µF
k = 0.50 for Ck = 222.20µF
k = 0.55 for Ck = 277.75µF
k = 0.60 for Ck = 333.30µF
k = 0.63 for Ck = 388.85µF
k = 0.65 for Ck = 444.40µF
k = 0.68 for Ck = 499.95µF
The valued obtained for C given diferent Ck's is:
C = 2696788.26µF for Ck = 22.22µF
C = -581202.19µF for Ck = 55.55µF
C = 213.49µF for Ck = 111.10µF
C = 231.23µF for Ck = 166.65µF
C = 225.28µF for Ck = 222.20µF
C = 228.66µF for Ck = 277.75µF
C = 222.20µF for Ck = 333.30µF
C = 232.82µF for Ck = 388.85µF
C = 238.55µF for Ck = 444.40µF
C = 240.06µF for Ck = 499.95µF
```

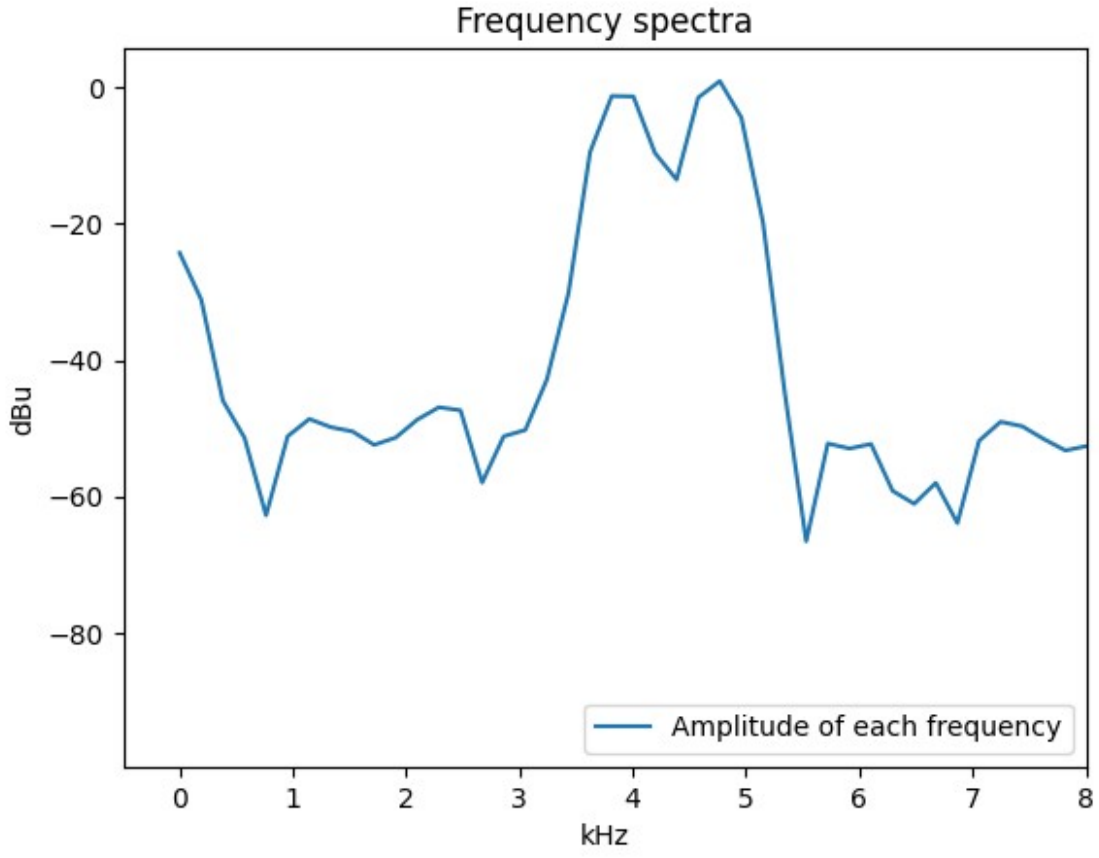


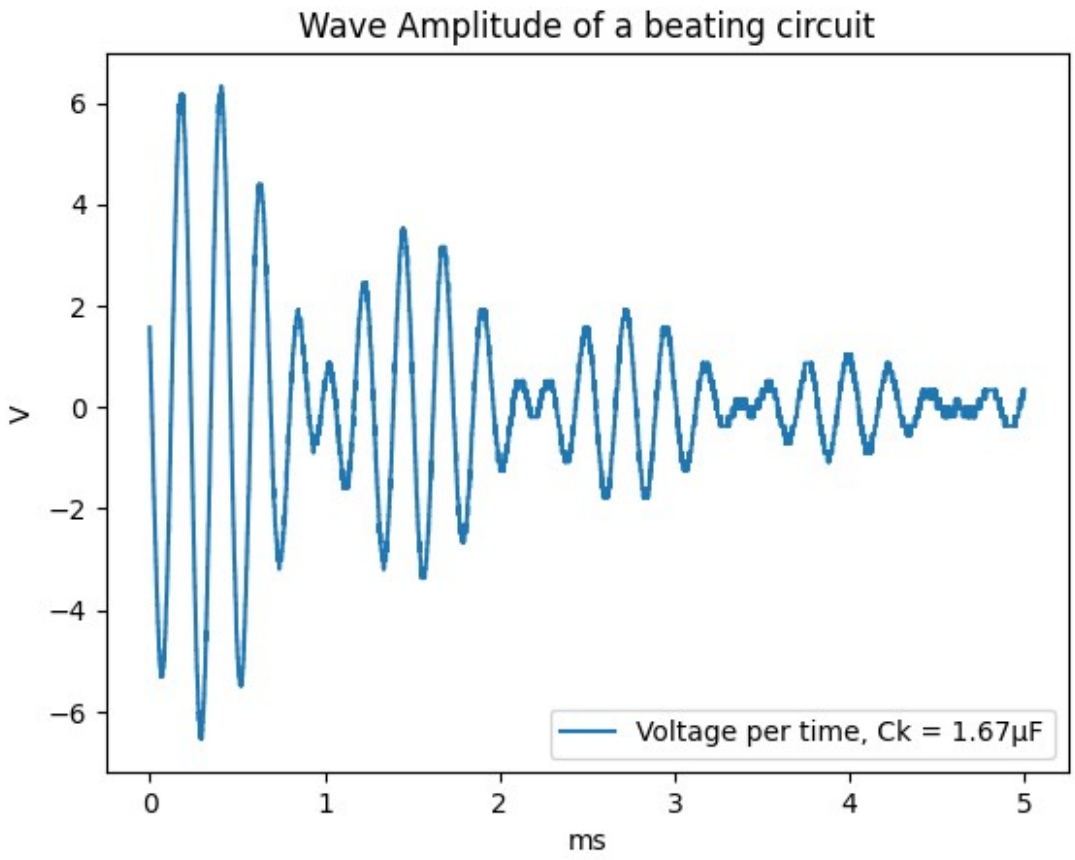
1. **Beat Measurements**
   - In the high-point circuit, short one branch, charge the other, then close the switch.
   - Acquire both time-trace and spectrum of the beat oscillation.

```
#data import
data1 = pd.read_csv('Data/Task3/Task3_freq_1.csv', sep=',')
freq = data1['Frequency'].to_numpy()
Af = data1['Channel A'].to_numpy()

data1 = pd.read_csv('Data/Task3/Task3_A_1.csv', sep=',')

#filtering other frequencies
x = Af > -2
rfrec = freq[x]

#taking the mean values of the peeks
f1 = (rfrec[0]+rfrec[1])/2
f2 = (rfrec[2]+rfrec[3])/2

#calculating the period
T = 2*np.pi/(f2-f1)

#result for T
```

```
print(f'The period of the beating is: {"%.2f"%T} ms')

#plotting
plt.figure(1)
plt.xlim(-0.5,8)
plt.title('Frequency spectra')
plt.xlabel('kHz')
plt.ylabel('dBu')
plt.plot(freq,Af, label='Amplitude of each frequency')
plt.legend(loc = 'lower right')

plt.figure(2)
plt.title('Wave Amplitude of a beating circuit')
plt.xlabel('ms')
plt.ylabel('V')
plt.plot(data1['Time'], data1['Channel A'], label=f'Voltage per time,
Ck = {"%.2f"%(0.015*1.111e2)}μF')
plt.legend(loc='lower right')
plt.show()
```

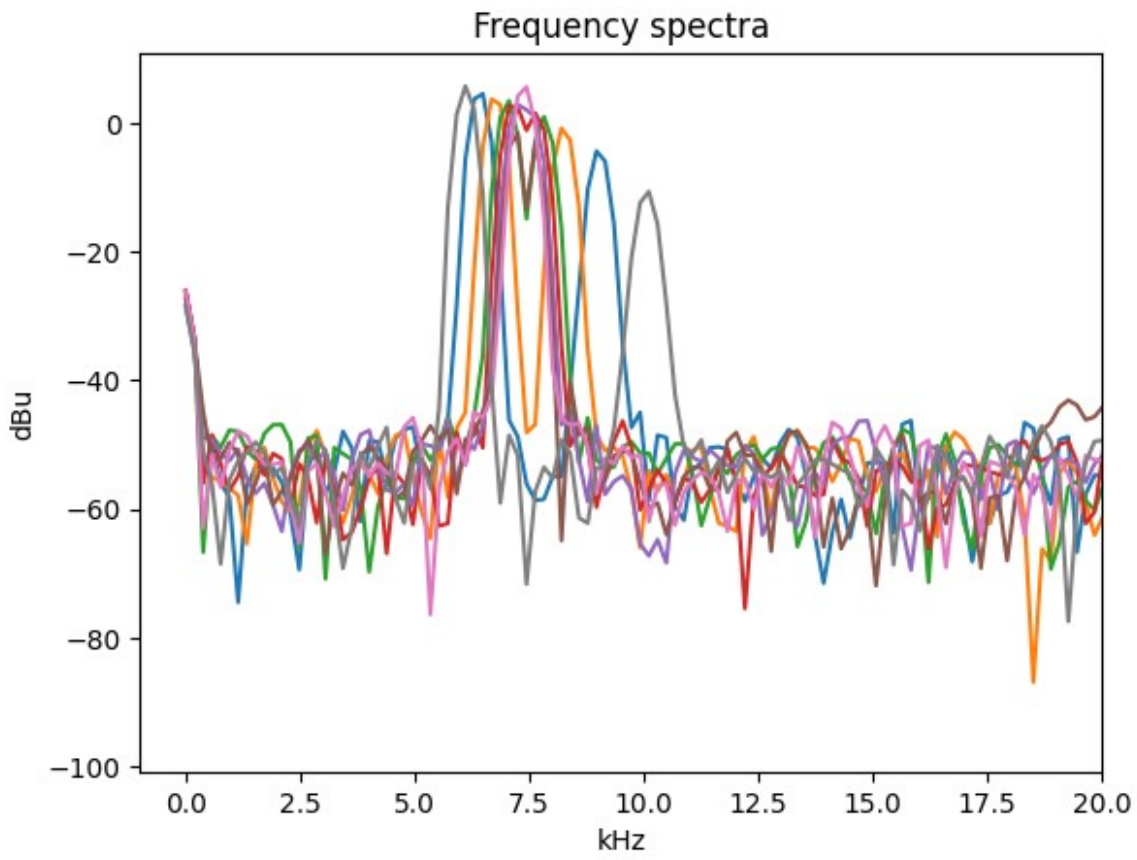
```
The period of the beating is: 8.24 ms
```





1. **Inductive Coupling**
   - Use two identical flat coils facing each other.
   - ary the coil separation d over eight positions.
   - For each d, measure uncoupled f_0 and coupled $f_\pm$ to compute M and K_L.

```
A_list = []

for i in range(1,9):
    data1 = pd.read_csv(f'Data/Task4/Task4_freq_{i}.csv', sep=',')
    freq = data1['Frequency'].to_numpy()
    y = data1['Channel A'].to_numpy()
    A_list.append(y)

A = np.vstack(A_list)

for i in range(0,8):
    plt.title('Frequency spectra')
    plt.xlabel('kHz')
    plt.ylabel('dBu')
    plt.xlim(-1,20)
    plt.plot(freq, A[i])
```



All instrument settings (time base, trigger level, FFT parameters) are optimized to resolve both f_1 and f_2 clearly. Data are logged systematically for subsequent plotting and fitting.